

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT VĨNH LONG

KHÓA LUẬN TỐT NGHIỆP
NGÀNH CÔNG NGHỆ THÔNG TIN

**ĐỀ TÀI: XÂY DỰNG HỆ THỐNG GIÁM SÁT
VÀ QUẢN LÝ BÃI ĐỖ XE THÔNG MINH
ỨNG DỤNG CÔNG NGHỆ AIOT
VÀ TRÍ TUỆ NHÂN TẠO**

CBHD: PGS.TS Phan Anh Cang

Sinh viên thực hiện: Trần Văn Còn

Mã số sinh viên: 22004015

Vĩnh Long, năm 2026



BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT VĨNH LONG

KHOA CÔNG NGHỆ THÔNG TIN

KHÓA LUẬN TỐT NGHIỆP

NGÀNH CÔNG NGHỆ THÔNG TIN

**ĐỀ TÀI: XÂY DỰNG HỆ THỐNG GIÁM SÁT
VÀ QUẢN LÝ BÃI ĐỖ XE THÔNG MINH
ỨNG DỤNG CÔNG NGHỆ AIOT
VÀ TRÍ TUỆ NHÂN TẠO**

CBHD: PGS.TS Phan Anh Cang

Sinh viên thực hiện: Trần Văn Còn

Mã số sinh viên: 22004015

Vĩnh Long, năm 2026

PHIẾU GIAO KHÓA LUẬN TỐT NGHIỆP

Tên đề tài: ***Xây dựng hệ thống giám sát và quản lý bãi đỗ xe thông minh ứng dụng công nghệ AIoT và trí tuệ nhân tạo.***

- Nghiên cứu công nghệ định vị Ultra-Wideband (UWB), mô hình Anchor-Tag và thuật toán Trilateration trong bài toán xác định vị trí phương tiện ở không gian trong nhà; trên cơ sở đó thiết kế và lập trình firmware cho module UWB EWT550 kết hợp vi điều khiển ESP32-C3 (đảm nhận hai vai trò Anchor và Tag) nhằm thu thập và truyền dữ liệu khoảng cách theo thời gian thực qua UART và MQTT.
- Xây dựng IoT Server đảm nhiệm việc xử lý dữ liệu cảm biến, tính toán tọa độ phương tiện, lọc nhiễu kết quả định vị, lưu trữ lịch sử di chuyển và phân phối trạng thái theo thời gian thực.
- Xây dựng ứng dụng di động trên nền tảng React Native/Expo cho phép hiển thị bản đồ bãi đỗ xe cùng vị trí và trạng thái phương tiện được cập nhật theo thời gian thực.
- Nghiên cứu và triển khai chatbot hỏi đáp tiếng Việt ứng dụng kỹ thuật Retrieval-Augmented Generation (RAG), kết hợp truy xuất ngữ nghĩa và truy xuất từ khóa nhằm hỗ trợ tra cứu thông tin về hệ thống bãi đỗ xe.
- Tiến hành kiểm thử chức năng hệ thống và đánh giá độ chính xác của kết quả định vị trong điều kiện thực nghiệm.

Phương pháp đánh giá: ☐ Báo cáo trước hội đồng ☐ Chấm thuyết minh

Ngày giao khóa luận: ngày ... tháng ... năm 2026

Ngày hoàn thành khóa luận: ngày ... tháng ... năm 2026

Số lượng sinh viên thực hiện khóa luận: 1

Họ và tên sinh viên: Trần Văn Còn..... MSSV: 22004015

Vĩnh Long, ngày tháng năm 2026

Trưởng Khoa/Bộ môn

Người hướng dẫn

Phan Anh Cang

PGS.TS Phan Anh Cang

NHẬN XÉT & ĐÁNH GIÁ CỦA GV HƯỚNG DẪN

Ý thức thực hiện:

.....

.....

Nội dung thực hiện:

.....

.....

Hình thức trình bày:

.....

.....

Tổng hợp kết quả:

☐ Tổ chức báo cáo trước hội đồng

☐ Tổ chức chấm thuyết minh

Vĩnh Long, ngày tháng năm

Người hướng dẫn

PGS.TS Phan Anh Cang

LỜI CẢM ƠN

Khóa luận tốt nghiệp với đề tài "Xây dựng hệ thống giám sát và quản lý bãi đỗ xe thông minh ứng dụng công nghệ AIoT và trí tuệ nhân tạo" là kết quả của quá trình học tập và nghiên cứu của em tại Khoa Công nghệ Thông tin — Trường Đại học Sư phạm Kỹ thuật Vĩnh Long. Để hoàn thành khóa luận này, em đã nhận được sự hướng dẫn và hỗ trợ tận tình từ quý thầy cô.

Em xin gửi lời cảm ơn sâu sắc đến PGS.TS. Phan Anh Cang — giảng viên trực tiếp hướng dẫn em thực hiện đề tài. Thầy đã định hướng nghiên cứu và đưa ra những nhận xét chuyên môn quan trọng trong từng giai đoạn, từ thiết kế phần cứng định vị UWB, xây dựng IoT Server xử lý dữ liệu đến phát triển hệ thống chatbot hỏi đáp. Sự góp ý của thầy đã giúp em khắc phục nhiều hạn chế và hoàn thiện đề tài.

Em cũng xin cảm ơn quý thầy cô trong Khoa Công nghệ Thông tin đã trang bị cho em nền tảng kiến thức chuyên môn trong suốt quá trình học tập, tạo điều kiện để em tiếp cận và thực hiện một đề tài có tính ứng dụng cao.

Em xin gửi lời cảm ơn đến gia đình và bạn bè đã luôn ủng hộ, động viên em trong thời gian thực hiện khóa luận.

Do kiến thức và kinh nghiệm thực tiễn còn hạn chế, khóa luận không tránh khỏi những thiếu sót. Em rất mong nhận được ý kiến đóng góp của quý thầy cô để đề tài được hoàn thiện hơn.

Em xin chân thành cảm ơn!

MỤC LỤC

NHẬN XÉT & ĐÁNH GIÁ CỦA GV HƯỚNG DẪN.....	i
LỜI CẢM ƠN.....	ii
MỤC LỤC.....	iii
DANH MỤC TỪ VIẾT TẮT.....	vii
DANH MỤC HÌNH ẢNH.....	viii
DANH MỤC BẢNG.....	x
LỜI NÓI ĐẦU.....	2
TÓM TẮT.....	4
CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI.....	5
1.1 Lý do chọn đề tài.....	5
1.2 Mục tiêu nghiên cứu.....	6
1.3 Đối tượng và phạm vi nghiên cứu.....	7
1.4 Phương pháp nghiên cứu.....	7
1.5 Ý nghĩa khoa học và thực tiễn.....	7
1.6 Các nghiên cứu liên quan.....	9
1.7 Cấu trúc khóa luận.....	10
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.....	12
2.1 Công nghệ định vị trong nhà Ultra-Wideband (UWB).....	12
2.1.1 So sánh với các công nghệ định vị khác.....	13
2.1.2 Cách đo khoảng cách: Two-Way Ranging.....	13
2.1.3 Xác định vị trí 2D: Trilateration.....	14
2.2 Bộ lọc One-Euro.....	15
2.3 Vi điều khiển ESP32-C3.....	16

2.4 Bluetooth Low Energy (BLE).....	17
2.5 Giao thức MQTT.....	18
2.6 NestJS.....	19
2.7 Cơ sở dữ liệu PostgreSQL và pgvector.....	20
2.8 Redis.....	iv
2.9 React Native và Expo.....	21
2.10 Server-Sent Events (SSE) và Socket.IO.....	22
2.11 Kỹ thuật Hybrid RAG (Retrieval-Augmented Generation).....	22
2.12 Giao thức MCP (Model Context Protocol).....	26
2.13 Docker và Nginx.....	27
2.14 Mô hình ngôn ngữ lớn (LLM) và nền tảng chatbot.....	28
CHƯƠNG 3: PHƯƠNG PHÁP ĐỀ XUẤT.....	30
3.1 Mô tả tổng quan hệ thống đề xuất.....	30
3.2 Thiết kế phần cứng và Firmware.....	31
3.3 Quy trình xử lý tại IoT Server.....	33
3.4 Cơ chế Hybrid RAG cho Gara thông minh.....	34
3.5 Quy trình hoạt động: Xe vào — Xe ra.....	35
3.5.1 Giai đoạn 1: Xe vào bãi.....	36
3.5.2 Giai đoạn 2: Định vị liên tục trong bãi.....	37
3.5.3 Giai đoạn 3: Giám sát và cảnh báo.....	38
3.5.4 Giai đoạn 4: Xe ra khỏi ô đỗ.....	38
3.5.5 Giai đoạn 5: Check-out và đóng lượt đỗ.....	39
3.6 Tích hợp Hệ thống Quản lý Gara (Bên thứ ba).....	40
CHƯƠNG 4: KẾT QUẢ THỰC HIỆN.....	41
4.1 Môi trường triển khai.....	41

4.2 Tính năng ứng dụng di động.....	42
4.2.1 Bản đồ bãi đỗ xe 3D Isometric thời gian thực.....	42
4.2.2 Quản lý xe và QR Code.....	42
4.2.3 Tìm xe (Find My Car).....	43
4.2.4 Khảo sát mặt bằng (Floor Survey — Walk & Press).....	44
4.2.5 Phát hiện đỗ sai quy định (Mispark Detection).....	44
4.2.6 Cảnh báo an ninh: chống trộm và vùng giới hạn (geofence).....	44
4.2.7 Chế độ Gara (Standard / Valet).....	45
4.2.8 Đề xuất bố trí tự động từ dữ liệu vận hành (ô đỗ và lối đi).....	45
4.2.9 Thông báo đẩy và thông báo trong ứng dụng.....	46
4.2.10 Giao diện người dùng.....	46
4.3 Tính năng Chatbot AI.....	47
4.3.1 Hội thoại tự nhiên bằng tiếng Việt.....	47
4.3.2 Agentic Loop và MCP Tools.....	48
4.3.3 Thẻ giao diện trực quan.....	48
4.3.4 Lịch sử hội thoại đa phiên.....	49
4.3.5 Dual-Provider LLM với Fallback.....	49
4.3.6 Đánh giá chất lượng truy hồi (Hybrid RAG).....	49
4.3.7 Đánh giá chất lượng trả lời theo môi trường (none / MCP / MCP+RAG)	50
4.4 Tính năng IoT Admin (Web Dashboard).....	52
4.5 Pipeline xử lý dữ liệu định vị thời gian thực.....	52
4.5.1 Tổng quan đường ống xử lý.....	53
4.5.2 Trilateration không trạng thái và loại bỏ lời giải nhiễu.....	54
4.5.3 Làm tròn, tiết lưu cập nhật và phát hiện mất tín hiệu.....	55

4.5.4 Kiểm chứng vận hành trên môi trường mô phỏng (Sandbox).....	55
4.6 Phương pháp đánh giá độ chính xác định vị UWB.....	56
4.6.1 Bố trí thực nghiệm.....	56
4.6.2 Quy trình và chỉ tiêu đo.....	57
4.6.3 Định nghĩa hình thức các độ đo sai số định vị.....	58
4.6.4 Kết quả kiểm chứng quy trình trên dữ liệu mô phỏng.....	60
4.6.5 Ngưỡng cảnh hóa độ chính xác kỳ vọng theo tài liệu.....	61
4.6.6 Kiến trúc pipeline định vị thời gian thực.....	62
4.7 Phân tích và khai thác dữ liệu vận hành.....	63
4.8 Phân tích và khai thác dữ liệu vận hành.....	64
4.9 Đánh giá hợp đồng công cụ – thẻ giao diện.....	66
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	68
5.1 Kết luận.....	68
5.2 Hạn chế của đề tài.....	68
5.3 Hướng phát triển tương lai.....	69
TÀI LIỆU THAM KHẢO.....	70

DANH MỤC TỪ VIẾT TẮT

STT	Từ viết tắt	Nội dung đầy đủ
1	AIoT	Artificial Intelligence of Things
2	UWB	Ultra-Wideband
3	TWR	Two-Way Ranging
4	MQTT	Message Queuing Telemetry Transport
5	TLS	Transport Layer Security
6	RAG	Retrieval-Augmented Generation
7	BM25	Best Match 25
8	RRF	Reciprocal Rank Fusion
9	SSE	Server-Sent Events
10	LLM	Large Language Model
11	NLP	Natural Language Processing
12	GPS	Global Positioning System
13	MCP	Model Context Protocol
14	TTL	Time To Live
15	API	Application Programming Interface
16	ORM	Object-Relational Mapping
17	GIN	Generalized Inverted Index
18	RSSI	Received Signal Strength Indicator
19	NLOS	Non-Line-of-Sight
20	LOS	Line-of-Sight

DANH MỤC HÌNH ẢNH

Hình 2.1: Công nghệ Ultra-Wideband (UWB).....	12
Hình 2.2: Two-Way Ranging (TWR).....	14
Hình 2.3: Nguyên lý định vị Trilateration.....	15
Hình 2.4: Bộ lọc One-Euro.....	16
Hình 2.5: ESP32-C3 Super Mini.....	17
Hình 2.6: Logo giao thức MQTT.....	18
Hình 2.7: Logo NestJS.....	19
Hình 2.8: Logo PostgreSQL.....	20
Hình 2.9: Logo Redis.....	21
Hình 2.10: Logo React Native và Expo.....	21
Hình 2.11: Mô hình Retrieval-Augmented Generation (RAG).....	23
Hình 2.12: Kiến trúc Hybrid RAG tích hợp IoT Snapshot.....	24
Hình 2.13: Model Context Protocol (MCP).....	27
Hình 2.14: Kiến trúc triển khai Docker + Nginx.....	28
Hình 3.1: Kiến trúc tổng thể hệ thống AIoT.....	31
Hình 3.2: Sơ đồ đấu nối mạch phần cứng nút UWB.....	31
Hình 3.3: Một nút UWB thực tế đã lắp ráp (ESP32-C3 Super Mini ghép module UWB EWT550) — dùng chung cho cả Anchor và Tag, chỉ khác cấu hình firmware.....	32
Hình 3.4: Quy trình khảo sát bãi đỗ (walk-and-press).....	33
Hình 3.5: Luồng dữ liệu định vị UWB.....	34
Hình 3.6: Luồng Chatbot AI.....	35
Hình 3.7: Lược đồ tuần tự tổng quan xe vào – xe ra.....	36

Hình 3.8: Giai đoạn 1 — Xe vào bãi và liên kết Tag.....	37
Hình 3.9: Giai đoạn 2 — Định vị liên tục trong bãi.....	37
Hình 3.10: Giai đoạn 3 — Giám sát và cảnh báo.....	38
Hình 3.11: Giai đoạn 4 — Xe ra khỏi ô đỗ và tìm xe.....	39
Hình 3.12: Giai đoạn 5 — Check-out và đóng lượt đỗ.....	39
Hình 4.1: Giao diện bản đồ định vị 3D trên ứng dụng.....	42
Hình 4.2: Giải pháp tìm xe bằng Tag UWB cầm tay.....	43
Hình 4.3: Màn hình chatbot AI tiếng Việt.....	48
Hình 4.4: Mô phỏng Sandbox — kịch bản nhiều xe (mặt bằng hình L).....	56
Hình 4.5: Bố trí thực nghiệm đánh giá định vị UWB.....	57
Hình 4.6: Quy trình phân tích dữ liệu vận hành.....	65

DANH MỤC BẢNG

Bảng 2.1: So sánh các công nghệ định vị trong nhà.....	13
Bảng 4.1: So sánh độ chính xác và chi phí token của chatbot theo môi trường (DeepSeek-V3.2 chat, chấm tự động khách quan, n=101).....	50
Bảng 4.2: Tham số thiết kế của pipeline định vị thời gian thực (đối chiếu trực tiếp thông số thiết kế của mô-đun xử lý luồng vị trí).....	53
Bảng 4.3: Kết quả định vị trên dữ liệu mô phỏng (minh hoạ quy trình — CHƯA phải đo thực địa), chạy qua đúng đường ống trilateration + One-Euro (tần số cắt tối thiểu 0,5 Hz; $\beta = 0,1$).....	60
Bảng 4.4: Độ chính xác định vị điển hình của các công nghệ theo tài liệu.....	61
Bảng 4.5: Kết quả đo định vị thực địa (khung chờ số liệu — sẽ cập nhật khi có dữ liệu đo).....	64
Bảng 4.6: Kết quả kiểm thử hợp đồng công cụ – thẻ giao diện.....	66

LỜI NÓI ĐẦU

Trong những năm gần đây, sự kết hợp giữa Internet vạn vật (IoT) và trí tuệ nhân tạo (AIoT) đã mở ra hướng tiếp cận mới cho bài toán quản lý không gian đô thị. Bãi đỗ xe trong nhà là một trong những môi trường điển hình thụ hưởng xu hướng này, song cũng đặt ra thách thức riêng: tín hiệu định vị vệ tinh (GNSS) bị suy giảm, khiến việc xác định vị trí phương tiện theo thời gian thực trở thành bài toán khó.

Các hướng định vị trong nhà phổ biến đều bộc lộ hạn chế: GNSS gần như mất tác dụng do bị che chắn bởi kết cấu bê tông; Wi-Fi RSSI và Bluetooth Low Energy (BLE) cho chi phí thấp nhưng sai số lớn (thường ở mức vài mét) và kém ổn định trong điều kiện đa đường truyền (multipath); các giải pháp dựa trên camera lại phụ thuộc điều kiện ánh sáng và phát sinh lo ngại về quyền riêng tư. Xuất phát từ thực tế đó, khóa luận xây dựng Hệ thống giám sát và điều khiển Gara thông minh dựa trên công nghệ AIoT, lấy định vị Ultra-Wideband (UWB) làm nền tảng nhằm đạt sai số ở mức centimet.

Khóa luận trình bày các nội dung chính sau:

- Công nghệ định vị UWB, mô hình Anchor–Tag và thuật toán Trilateration.
- Firmware cho module UWB EWT550 trên vi điều khiển ESP32-C3, truyền dữ liệu qua UART và MQTT.
- IoT Server xử lý dữ liệu, tính tọa độ, lọc nhiễu và phân phối trạng thái theo thời gian thực.
- Ứng dụng di động React Native/Expo hiển thị bản đồ và trạng thái phương tiện.
- Chatbot hỏi đáp tiếng Việt ứng dụng kỹ thuật Retrieval-Augmented Generation (RAG).
- Kiểm thử chức năng và đánh giá độ chính xác định vị trong thực nghiệm.

Để trình bày một cách hệ thống các nội dung trên, khóa luận được tổ chức thành năm chương:

- **Chương 1: Tổng quan**
- **Chương 2: Cơ sở lý thuyết**
- **Chương 3: Phương pháp đề xuất**
- **Chương 4: Kết quả thực hiện**
- **Chương 5: Kết luận và hướng phát triển**

Tác giả xin chân thành cảm ơn PGS.TS. Phan Anh Cang cùng quý thầy cô Khoa Công nghệ Thông tin đã hướng dẫn và hỗ trợ trong quá trình thực hiện khóa luận.

TÓM TẮT

Khóa luận trình bày việc nghiên cứu và xây dựng Hệ thống giám sát và điều khiển Gara thông minh dựa trên công nghệ AIoT. Hệ thống sử dụng công nghệ định vị Ultra-Wideband (UWB) kết hợp thuật toán Trilateration và bộ lọc One-Euro để xác định vị trí phương tiện trong nhà theo thời gian thực, trên nền vi điều khiển ESP32-C3 và giao thức truyền thông MQTT. Dữ liệu khoảng cách được xử lý tại máy chủ IoT (IoT Server) để tính tọa độ, làm trơn quỹ đạo và phân phối tới ứng dụng di động, nơi vị trí phương tiện được hiển thị trực quan trên bản đồ số 3D. Bên cạnh đó, hệ thống tích hợp một chatbot tiếng Việt ứng dụng kỹ thuật Tăng cường Truy xuất (Retrieval-Augmented Generation – RAG) cho phép người dùng tra cứu trạng thái bãi đỗ và vị trí phương tiện bằng ngôn ngữ tự nhiên. Hệ thống đã được hiện thực hóa hoàn chỉnh từ phần cứng đến ứng dụng di động và giao diện quản trị web; độ chính xác định vị được đánh giá theo một quy trình định lượng chuẩn hóa dựa trên các độ đo RMSE, MAE và CEP.

Từ khóa: UWB, định vị trong nhà, Trilateration, bộ lọc One-Euro, AIoT, chatbot RAG, MQTT, ESP32-C3.

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

Chương 1 trình bày tổng quan về đề tài. Nội dung chương lần lượt làm rõ lý do chọn đề tài xuất phát từ nhu cầu định vị phương tiện trong không gian kín; mục tiêu, đối tượng và phạm vi nghiên cứu; phương pháp thực hiện; ý nghĩa khoa học và thực tiễn; các nghiên cứu liên quan trong lĩnh vực định vị trong nhà; và cuối cùng là cấu trúc tổng thể của khóa luận.

1.1 Lý do chọn đề tài

Trong bối cảnh đô thị hóa nhanh và số lượng phương tiện cá nhân không ngừng gia tăng, các bãi đỗ xe và gara sửa chữa trong nhà ngày càng phải tiếp nhận một lưu lượng phương tiện lớn trên một mặt bằng giới hạn. Khi quy mô vượt qua một ngưỡng nhất định, việc theo dõi vị trí của từng phương tiện bằng quan sát trực tiếp hay ghi chép thủ công trở nên kém khả thi: thao tác chậm, dễ nhầm lẫn và khó truy vết khi cần. Thực trạng này đặt ra yêu cầu về một phương thức giám sát có khả năng xác định vị trí từng phương tiện một cách tự động và chính xác.

Yêu cầu trên càng trở nên cấp thiết đối với gara sửa chữa nội bộ. Khác với bãi đỗ công cộng nơi mỗi xe gắn với một ô cố định, gara vận hành theo quy trình tiếp nhận – di chuyển – sửa chữa – bàn giao, trong đó một phương tiện có thể lần lượt đi qua nhiều khu vực như khu chờ, khoang sửa chữa và khu hoàn thiện. Các phương thức truyền thống như cảm biến đếm lượt ra – vào hay sơ đồ vị trí tĩnh chỉ phản ánh số lượng hoặc trạng thái cố định, không thể theo kịp sự dịch chuyển liên tục này. Do đó, hệ thống cần cung cấp thông tin vị trí cập nhật theo thời gian thực, vừa để xác định nhanh chỗ trống và định vị lại phương tiện khi bàn giao, vừa để lưu vết lịch sử di chuyển phục vụ đối soát và phát hiện các tình huống bất thường như đỗ sai khu vực hay lưu lại quá lâu.

Tuy nhiên, các công nghệ định vị phổ biến hiện nay đều bộc lộ hạn chế khi áp dụng trong môi trường kín. Tín hiệu định vị vệ tinh (GNSS) bị kết cấu bê tông che chắn nên gần như mất tác dụng trong nhà; định vị bằng Wi-Fi hoặc Bluetooth Low Energy (BLE) tuy chi phí thấp nhưng kém ổn định và dễ bị nhiễu bởi hiện tượng đa đường; trong khi giải pháp dựa trên camera lại phụ thuộc điều kiện ánh sáng và làm

này sinh lo ngại về quyền riêng tư. Trước những hạn chế đó, công nghệ băng siêu rộng Ultra-Wideband (UWB) nổi lên như một hướng tiếp cận phù hợp: nhờ do thời gian truyền của xung sóng có băng thông rất rộng, UWB phân biệt tốt tín hiệu đi thẳng với tín hiệu phản xạ và đạt được độ chính xác ở cấp xen-ti-mét ngay trong không gian nhiều vật cản. Đây chính là cơ sở để khóa luận lựa chọn UWB làm công nghệ định vị nền tảng.

Bên cạnh bài toán định vị, quá trình vận hành còn đặt ra nhu cầu tra cứu thông tin một cách thuận tiện. Việc tổng hợp tình trạng bãi đỗ, vị trí và lịch sử phương tiện thường đòi hỏi nhiều thao tác thủ công trên các màn hình khác nhau. Sự phát triển gần đây của các mô hình ngôn ngữ lớn kết hợp kỹ thuật Truy hồi – Tăng cường Sinh (Retrieval-Augmented Generation – RAG) mở ra khả năng xây dựng một trợ lý hỏi đáp bằng tiếng Việt, cho phép người vận hành truy vấn dữ liệu thực tế của hệ thống bằng ngôn ngữ tự nhiên. Đây được xem là lớp tiện ích bổ trợ, giúp khai thác hiệu quả hơn nguồn dữ liệu mà lớp định vị tạo ra.

Từ những phân tích trên, đề tài "***Hệ thống giám sát và điều khiển Gara thông minh dựa trên công nghệ AIoT***" được thực hiện, lấy định vị phương tiện chính xác trong nhà bằng công nghệ UWB làm trọng tâm, đồng thời tích hợp một trợ lý hỏi đáp ứng dụng trí tuệ nhân tạo nhằm hỗ trợ người dùng khai thác dữ liệu của hệ thống.

1.2 Mục tiêu nghiên cứu

Mục tiêu tổng quát: xây dựng Hệ thống giám sát và điều khiển Gara thông minh dựa trên công nghệ AIoT, có khả năng định vị phương tiện trong nhà theo thời gian thực và hỗ trợ tra cứu thông tin bằng chatbot AI.

Mục tiêu cụ thể:

1. Nghiên cứu và triển khai giải pháp định vị trong nhà dựa trên công nghệ UWB đạt độ chính xác ở mức xen-ti-mét.
2. Thiết kế và chế tạo mô hình phần cứng Tag và Anchor sử dụng vi điều khiển ESP32-C3.
3. Xây dựng hệ thống backend (IoT Server) xử lý dữ liệu định vị theo thời gian thực và quản lý trạng thái bãi xe.

4. Phát triển ứng dụng di động hiển thị vị trí xe trên bản đồ số và hỗ trợ tìm xe.
5. Tích hợp chatbot AI thông minh ứng dụng kỹ thuật RAG (Retrieval-Augmented Generation) để hỗ trợ người dùng truy vấn trạng thái garage bằng ngôn ngữ tự nhiên.

1.3 Đối tượng và phạm vi nghiên cứu

- **Đối tượng nghiên cứu:** công nghệ định vị vô tuyến UWB (Ultra-Wideband), thuật toán Trilateration, bộ lọc One-Euro, kiến trúc hệ thống IoT, kỹ thuật RAG và Hybrid Search trong trí tuệ nhân tạo.
- **Phạm vi nghiên cứu** được giới hạn theo ba phương diện: (i) về *nội dung* — xác định tọa độ 2D (x, y) của phương tiện (không xét độ cao z), sử dụng bốn Anchor cố định cho mỗi tầng và giao tiếp qua giao thức MQTT; (ii) về *không gian* — môi trường bãi đỗ xe/gara trong nhà, thử nghiệm trên mô hình mặt bằng một tầng; (iii) về *thời gian* — trong khuôn khổ thời gian thực hiện khóa luận tốt nghiệp năm 2026.

1.4 Phương pháp nghiên cứu

Để đạt được các mục tiêu đề ra, đề tài áp dụng các phương pháp sau:

- **Nghiên cứu lý thuyết:** Tổng hợp tài liệu khoa học về các phương pháp định vị trong nhà, các mô hình đo khoảng cách bằng sóng vô tuyến, thuật toán lọc nhiễu tín hiệu và kiến trúc mô hình ngôn ngữ lớn phục vụ xây dựng chatbot.
- **Thực nghiệm và triển khai:** Thiết kế phần cứng, lập trình firmware cho vi điều khiển nhúng, xây dựng hệ thống server xử lý dữ liệu và phát triển ứng dụng di động đa nền tảng.
- **Đánh giá và kiểm thử:** Tiến hành đo đạc thực tế tại bãi xe để đánh giá độ chính xác định vị và hiệu quả phản hồi của hệ thống chatbot AI.

1.5 Ý nghĩa khoa học và thực tiễn

Về **mặt khoa học**, đề tài hệ thống hóa và vận dụng một chuỗi kỹ thuật hiện đại cho bài toán định vị trong nhà: đo khoảng cách Two-Way Ranging trên nền UWB,

giải tọa độ bằng Trilateration theo phương pháp bình phương tối thiểu, và làm trơn quỹ đạo bằng bộ lọc One-Euro. Trên nền tảng định vị đó, đề tài bổ sung một lớp trợ lý hỏi đáp ứng dụng kỹ thuật Hybrid RAG để khai thác dữ liệu thời gian thực bằng ngôn ngữ tự nhiên. Việc kết hợp một lớp định vị độ chính xác cao với một lớp tương tác thông minh trong cùng một kiến trúc AIoT thống nhất là đóng góp mang tính tổng hợp của khóa luận.

Về mặt thực tiễn, hệ thống giải quyết các nhu cầu cụ thể trong vận hành gara: xác định nhanh vị trí phương tiện, giám sát hiện trạng lắp đầy, phát hiện đổ sai quy định và hỗ trợ tra cứu thông tin bằng tiếng Việt. Thiết kế dựa trên phần cứng chi phí thấp (vi điều khiển ESP32-C3 kết hợp module UWB giá khoảng 2 USD) cùng phần mềm mã nguồn mở giúp giải pháp có khả năng nhân rộng cho các bãi đỗ trong nhà ở nhiều quy mô khác nhau.

Đóng góp cụ thể của khóa luận gồm năm điểm chính, trong đó ba đóng góp đầu thuộc về lớp định vị – nền tảng trọng tâm của đề tài, và hai đóng góp sau thuộc lớp tích hợp và trợ lý AI bổ trợ:

1. **Pipeline định vị UWB thời gian thực**: xây dựng hoàn chỉnh chuỗi xử lý từ bản tin khoảng cách Tag–Anchor đến tọa độ phương tiện, gồm giải Trilateration bằng bình phương tối thiểu và làm trơn quỹ đạo bằng bộ lọc One-Euro thích nghi, đạt độ chính xác cấp xen-ti-mét trong điều kiện trực xạ.
2. **Quy trình khảo sát mặt bằng walk-and-press**: phương pháp thiết lập bản đồ tầng hầm bằng cách đi bộ một vòng với Tag UWB, không cần đo vẽ thủ công; server tự phát hiện tọa độ các Anchor và dựng polygon ranh giới bãi đỗ từ dữ liệu khoảng cách thu thập trong thực địa.
3. **Bản đồ bãi đỗ tự học**: từ lịch sử vị trí tích lũy, hệ thống tự suy luận bố trí ô đỗ (phân cụm mean-shift kết hợp PCA xác định hướng hàng) và lối đi (rời rạc hóa lưới mật độ và gom vùng 8-connectivity), cho phép bản đồ được tinh chỉnh liên tục từ dữ liệu vận hành mà không cần đo vẽ lại — khép kín vòng đời từ định vị đến bố trí.
4. **Kiến trúc AIoT tích hợp bất đối xứng**: IoT Server xử lý toàn bộ dữ liệu định vị thô (trilateration, lọc nhiễu) rồi đẩy kết quả sạch qua Socket.IO cho hệ thống

quản lý gara của bên thứ ba; hai hệ thống vận hành độc lập, tạo thành một nền tảng end-to-end liên mạch từ firmware nhúng (ESP32-C3/EWT550) qua hạ tầng MQTT/NestJS/Redis/PostgreSQL đến ứng dụng di động và giao diện quản trị web.

5. **Mẫu kiến trúc "IoT Snapshot Documents" cho trợ lý hỏi đáp:** như một đóng góp bổ trợ ở lớp AI, đề tài đề xuất cách đưa dữ liệu IoT thời gian thực vào kho tri thức vector của hệ thống RAG — ảnh chụp trạng thái gara được nhúng và cập nhật định kỳ, cho phép trợ lý trả lời câu hỏi về trạng thái thực tế mà không cần gọi API riêng. Hiệu quả được kiểm chứng định lượng tại §4.3.6.

1.6 Các nghiên cứu liên quan

Định vị trong nhà là lĩnh vực đã được nghiên cứu rộng rãi. Khảo sát tổng quan của Zafari và cộng sự [6] phân loại các hệ thống định vị trong nhà theo công nghệ (Wi-Fi, BLE, UWB, RFID, âm thanh, từ trường) và theo phương pháp đo (RSSI, thời gian bay, góc tới, fingerprinting), đồng thời chỉ ra rằng các phương pháp dựa trên cường độ tín hiệu (RSSI) chỉ đạt sai số ở mức vài mét do nhạy với hiện tượng đa đường. Trong nhóm công nghệ băng siêu rộng, Che và cộng sự [7] xây dựng hệ thống định vị UWB cho môi trường Internet vạn vật công nghiệp (IIoT) sử dụng thuật toán Trilateration và bộ lọc Kalman để ổn định quỹ đạo, đạt độ chính xác ở cấp xen-ti-mét, khẳng định ưu thế của đo thời gian bay so với RSSI. Tuy nhiên, hệ thống trong [7] được thiết kế cho môi trường công nghiệp có hình học cố định và không tích hợp lớp giao diện người dùng hay chatbot AI. Đề tài này mở rộng hướng tiếp cận đó cho bài toán gara: bổ sung lớp khảo sát mặt bằng tự động (walk-and-press), tích hợp ứng dụng di động và chatbot RAG thành một kiến trúc AIoT thống nhất.

Ở khâu lọc nhiễu tín hiệu định vị, hai hướng tiếp cận phổ biến là bộ lọc Kalman [12] và bộ lọc One-Euro [2]. Kalman Filter là bộ lọc tối ưu tuyến tính cho hệ thống có mô hình động học (state-space model) đã biết — phù hợp khi quỹ đạo chuyển động có thể mô hình hóa (robot, phương tiện đường trơn). One-Euro Filter [2] không yêu cầu mô hình động học: tự động điều chỉnh mức độ lọc theo tốc độ thay đổi của tín hiệu — lọc mạnh khi tín hiệu thay đổi chậm (xe đứng yên), lọc nhẹ khi tín hiệu

thay đổi nhanh (xe di chuyển). Đối với bài toán bãi đỗ xe, quỹ đạo phương tiện không dự đoán được trước (có thể dừng đột ngột, vào bất kỳ ô nào) nên One-Euro phù hợp hơn: đơn giản hơn, không cần hiệu chỉnh ma trận nhiều, và có độ trễ thấp hơn trong các thử nghiệm tương tác [2].

Công trình	Phương pháp	Ưu/Hạn chế	Khác biệt với đề tài
Zafari et al. [6] (khảo sát)	Tổng hợp Wi-Fi/BLE/UWB/ RFID	Toàn diện nhưng không đề xuất hệ thống cụ thể	Đề tài triển khai hệ thống thực, không dừng ở khảo sát
Che et al. [7] (UWB-IIoT)	UWB Trilateration + Kalman filter	Chính xác cao, thiếu giao diện người dùng và AI	Thêm walk-and- press, ứng dụng di động và chatbot RAG
Lewis et al. [3] (RAG)	RAG với tài liệu tĩnh	Giảm hallucination, không có dữ liệu IoT thời gian thực	Đề tài mở rộng RAG với IoT Snapshot Documents
One-Euro vs Kalman [2][12]	Lọc tín hiệu thích nghi / tối ưu tuyến tính	One-Euro: đơn giản, độ trễ thấp; Kalman: cần mô hình động học	Đề tài chọn One- Euro vì quỹ đạo bãi đỗ không thể mô hình hóa trước

Ở lớp tương tác, kỹ thuật Retrieval-Augmented Generation do Lewis và cộng sự [3] đề xuất đã trở thành hướng tiếp cận phổ biến để chatbot truy cập tri thức bên ngoài và giảm hiện tượng "ảo giác" của mô hình ngôn ngữ lớn. Đề tài kế thừa hướng tiếp cận này và mở rộng cho dữ liệu IoT thời gian thực: thay vì chỉ truy xuất tài liệu tĩnh, hệ thống đưa trạng thái bãi đỗ và vị trí phương tiện cập nhật liên tục vào ngữ cảnh truy vấn.

1.7 Cấu trúc khóa luận

Khóa luận được tổ chức thành năm chương:

- **Chương 1 — Tổng quan đề tài:** trình bày lý do chọn đề tài, mục tiêu, đối tượng và phạm vi, phương pháp nghiên cứu, ý nghĩa khoa học – thực tiễn và các nghiên cứu liên quan.
- **Chương 2 — Cơ sở lý thuyết:** giới thiệu các công nghệ cốt lõi gồm UWB, Trilateration, bộ lọc One-Euro, vi điều khiển ESP32-C3, giao thức MQTT, NestJS, PostgreSQL/pgvector, Redis, React Native, RAG/MCP và Docker/Nginx.
- **Chương 3 — Phương pháp đề xuất:** mô tả kiến trúc tổng thể, thiết kế phần cứng – firmware, quy trình xử lý tại IoT Server, cơ chế Hybrid RAG và quy trình vận hành xe vào – xe ra.
- **Chương 4 — Kết quả thực hiện:** trình bày môi trường triển khai, các nhóm tính năng đã hiện thực hóa, phương pháp đánh giá độ chính xác định vị và phân tích dữ liệu vận hành.
- **Chương 5 — Kết luận và hướng phát triển:** tổng kết kết quả đạt được, nêu hạn chế và đề xuất hướng phát triển trong tương lai.

Kết luận chương 1. Chương 1 đã xác lập bối cảnh và động cơ của đề tài, làm rõ mục tiêu, đối tượng, phạm vi và phương pháp nghiên cứu, đồng thời nêu ý nghĩa khoa học – thực tiễn và đặt đề tài trong tương quan với các nghiên cứu liên quan. Các cơ sở lý thuyết phục vụ cho thiết kế hệ thống sẽ được trình bày trong Chương 2.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

Chương này giới thiệu các công nghệ cốt lõi được sử dụng trong hệ thống, bao gồm nguyên lý hoạt động, ưu nhược điểm và lý do lựa chọn. Các công nghệ được phân nhóm theo vai trò: lớp phần cứng IoT, lớp truyền thông, lớp xử lý backend, lớp giao diện người dùng và lớp AI.

2.1 Công nghệ định vị trong nhà Ultra-Wideband (UWB)



Hình 2.1: Công nghệ Ultra-Wideband (UWB)

Ultra-Wideband (UWB) là công nghệ vô tuyến sử dụng dải tần số rất rộng (từ 3,1–10,6 GHz, băng thông tối thiểu 500 MHz) và truyền các xung điện từ cực ngắn thay vì tín hiệu liên tục. Điểm khác biệt căn bản so với Wi-Fi hay Bluetooth là UWB đo **thời gian bay** (Time of Flight) của tín hiệu thay vì cường độ sóng. Vì tốc độ ánh sáng là hằng số đã biết, thời gian bay cho phép tính khoảng cách với độ chính xác 10–30 cm — vượt trội so với 1–5 m của các công nghệ dựa trên RSSI [6]. Trong môi trường bãi đỗ xe tầng hầm, nơi sóng GPS không xuyên qua bê tông và sóng Wi-Fi bị phản xạ nhiều từ xe kim loại, UWB là lựa chọn thực tế duy nhất đáp ứng được yêu cầu phân biệt chính xác từng ô đỗ.

2.1.1 So sánh với các công nghệ định vị khác

Bảng 2.1 so sánh ba công nghệ định vị phổ biến theo các tiêu chí quan trọng với bài toán định vị trong bãi đỗ xe trong nhà.

Bảng 2.1: So sánh các công nghệ định vị trong nhà

Tiêu chí	GPS	Wi-Fi / BLE	UWB
Nguyên lý đo	Tín hiệu vệ tinh	Cường độ sóng (RSSI)	Thời gian bay tín hiệu (ToF)
Hoạt động trong nhà	Không	Có	Có
Sai số điển hình	Không áp dụng	1 – 5 m	10 – 30 cm
Ảnh hưởng phản xạ đa đường	Cao	Rất cao	Thấp
Tiêu thụ điện	Thấp	Thấp	Thấp – Trung bình
Chi phí module	Thấp	Thấp	Trung bình (~2 USD)

2.1.2 Cách đo khoảng cách: Two-Way Ranging

Two-Way Ranging (TWR) là giao thức đo khoảng cách hai chiều giữa hai thiết bị UWB, được chuẩn hóa cùng các kỹ thuật đo khoảng cách nâng cao trong tiêu chuẩn IEEE 802.15.4z [1]. Luồng hoạt động như sau: thiết bị chủ động (Initiator) gửi một tín hiệu ping và ghi nhận thời điểm gửi; thiết bị bị động (Responder) nhận được, chờ một khoảng thời gian xử lý nội bộ cố định rồi phản hồi; Initiator đo tổng thời gian khứ hồi và tính ra khoảng cách. Ưu điểm quan trọng của TWR là **không cần đồng bộ đồng hồ** giữa hai thiết bị — điểm yếu lớn của phương pháp đo thời gian một chiều (TDoA).

Ảnh xạ vai trò trong hệ thống đề tài: Tag (module gắn trên xe, địa chỉ AB00) đóng vai trò **Initiator** theo thuật ngữ TWR — khởi động chu trình đo, thu thập khoảng cách đến tất cả Anchor trong một lần đo và gửi bản tin MQTT bản tin định vị gửi qua MQTT về server mỗi giây. Anchor (nút cố định AA01–AA04 tại các góc bãi) đóng

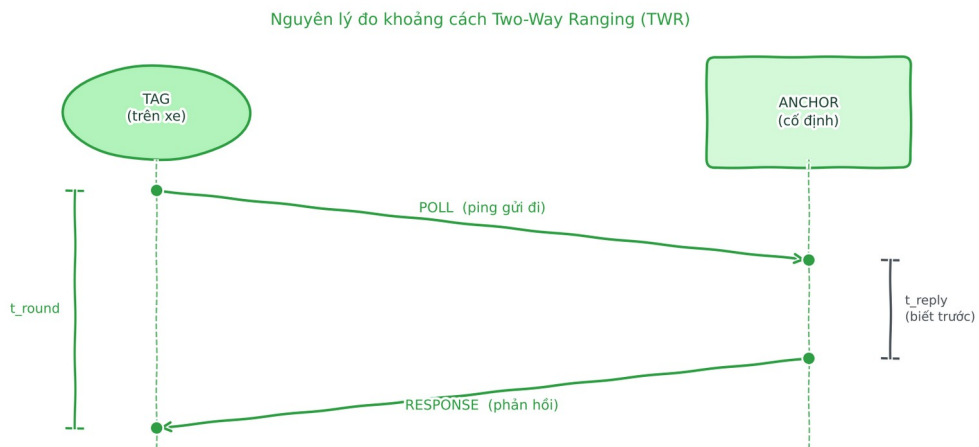
vai trò **Responder** — chỉ phản hồi tín hiệu từ Tag khi được poll, không tự khởi tạo đo khoảng cách và không gửi bản tin UWB độc lập lên server. Theo chuẩn EWT550, Initiator tương ứng với Base Station (ROLE=1), Responder tương ứng với Slave (ROLE=0).

Khoảng cách được tính theo công thức:

$$d = c \cdot \frac{t_{round} - t_{reply}}{2}$$

Trong đó:

- d — khoảng cách cần tính (m)
- c — tốc độ ánh sáng ($\approx 3 \times 10^8$ m/s)
- t_{round} — thời gian tổng từ lúc Initiator gửi đến lúc nhận được phản hồi (s)
- t_{reply} — thời gian Responder xử lý nội bộ trước khi phản hồi (s, đã biết trước)



Hình 2.2: Two-Way Ranging (TWR)

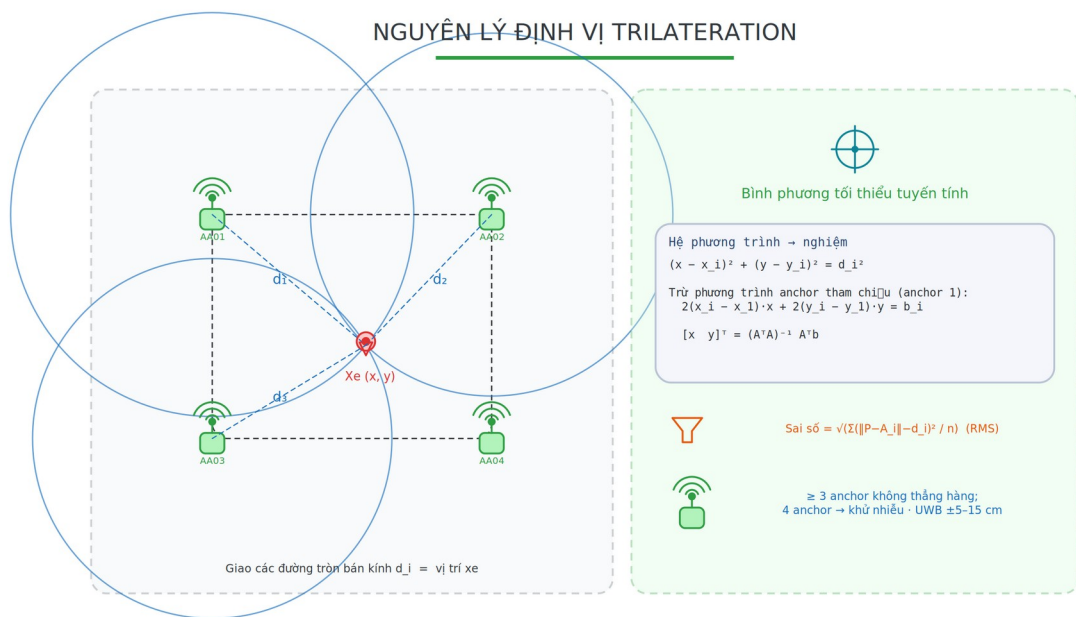
2.1.3 Xác định vị trí 2D: Trilateration

Sau khi đo được khoảng cách đến các Anchor, vị trí của Tag được xác định bằng thuật toán Trilateration. Ý tưởng: nếu biết Tag cách Anchor A_1 một khoảng d_1 , thì Tag nằm đâu đó trên đường tròn tâm A_1 bán kính d_1 . Vẽ đường tròn tương tự cho mỗi Anchor — giao điểm của tất cả các đường tròn cho vị trí duy nhất của Tag. Khi có nhiều điểm tham chiếu hơn mức tối thiểu, phương pháp Bình phương Tối thiểu (Least Squares) tìm nghiệm tốt nhất giảm thiểu tổng sai số đo:

$$p = (A^T A)^{-1} A^T b$$

Trong đó:

- p — vector tọa độ (x, y) ước tính của điểm cần tìm
- A — ma trận hệ số xây dựng từ tọa độ các điểm tham chiếu
- b — vector xây dựng từ khoảng cách đo được và tọa độ điểm tham chiếu
- $(\cdot)^T$ — phép chuyển vị ma trận
- $(\cdot)^{-1}$ — phép nghịch đảo ma trận



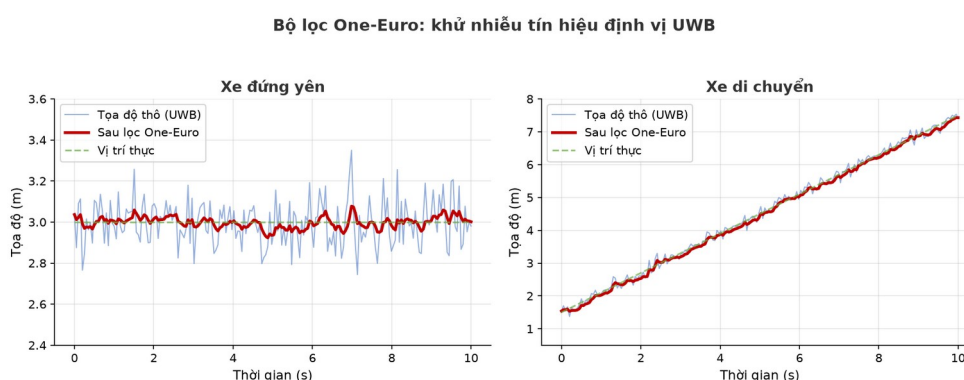
Hình 2.3: Nguyên lý định vị Trilateration

2.2 Bộ lọc One-Euro

Dù UWB chính xác hơn các công nghệ khác, tọa độ thô sau bước định vị vẫn dao động vài chục xen-ti-mét mỗi giây do nhiễu đo và phản xạ sóng, khiến vị trí xe "nhảy múa" liên tục trên bản đồ ngay cả khi xe đứng yên. Để khắc phục hiện tượng này, khóa luận sử dụng bộ lọc One-Euro [2] — một bộ lọc thông thấp thích nghi theo tốc độ. Nguyên lý của bộ lọc là tự điều chỉnh mức độ làm trơn theo tốc độ thay đổi của tín hiệu: khi tín hiệu biến đổi chậm, tương ứng với lúc xe đứng yên, bộ lọc tăng cường độ làm trơn để loại bỏ nhiễu; ngược lại, khi tín hiệu biến đổi nhanh, tương ứng

với lúc xe di chuyển, bộ lọc giảm làm trơn để bám kịp chuyển động thật và tránh gây trễ.

So với bộ lọc Kalman [12] vốn phổ biến trong điều khiển robot và thiết bị bay không người lái, One-Euro đơn giản hơn nhiều vì chỉ cần hai tham số và không đòi hỏi xây dựng mô hình động học của đối tượng. Đặc điểm này đặc biệt phù hợp với bài toán định vị xe trong bãi đỗ, nơi quỹ đạo di chuyển của phương tiện gần như không thể dự đoán trước.



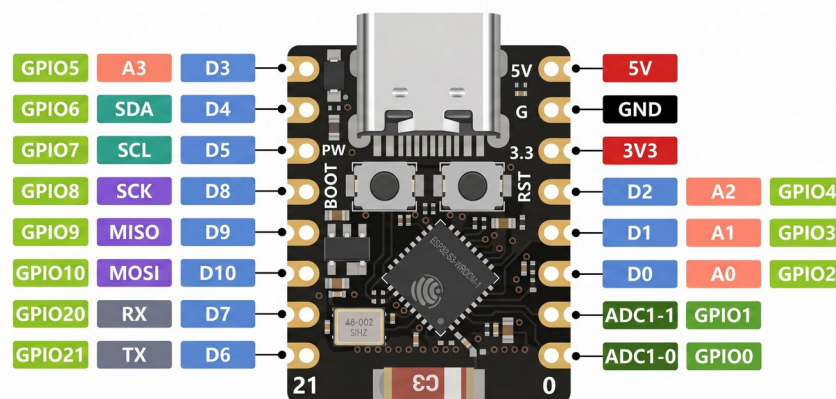
Hình 2.4: Bộ lọc One-Euro

2.3 Vi điều khiển ESP32-C3

ESP32-C3 là vi điều khiển tích hợp (SoC) do Espressif Systems (Trung Quốc) ra mắt năm 2020, là thế hệ kế tiếp của dòng ESP8266 và ESP32 nổi tiếng [13]. Espressif lựa chọn kiến trúc lõi RISC-V mã nguồn mở thay vì Xtensa trước đó, giúp tiết kiệm chi phí bản quyền và dễ tích hợp với chuỗi công cụ GCC nguồn mở. Chip tích hợp sẵn Wi-Fi 802.11 b/g/n và Bluetooth 5.0 BLE trên cùng một die, module Super Mini chỉ 18×22 mm với giá khoảng 1–2 USD.

Dòng ESP32 hiện là vi điều khiển IoT phổ biến nhất thế giới với hơn 1 tỷ chip đã xuất xưởng (tính đến 2024). Cộng đồng maker, sinh viên và kỹ sư toàn cầu đóng góp hàng nghìn thư viện, ví dụ và tutorial trên GitHub, Arduino Forum và Hackaday. ESP-IDF — framework chính thức của Espressif — cung cấp sẵn hệ điều hành thời gian thực FreeRTOS, stack Wi-Fi/BLE, cơ chế OTA và giao tiếp đa loại ngoại vi. Nhờ hội tụ khả năng kết nối mạng, chi phí thấp và kích thước nhỏ gọn, ESP32-C3 rất phù hợp để làm nút IoT nhúng như trong hệ thống của khóa luận; hạn chế của chip là

chỉ có một lõi xử lý, nên không thích hợp cho các tác vụ xử lý tín hiệu nặng hay chạy trí tuệ nhân tạo trực tiếp trên thiết bị.



Hình 2.5: ESP32-C3 Super Mini

2.4 Bluetooth Low Energy (BLE)

Bluetooth Low Energy (BLE) được giới thiệu trong chuẩn Bluetooth 4.0 năm 2010 bởi Bluetooth SIG, ban đầu mang tên Wibree (phát triển bởi Nokia từ 2006). Mục tiêu là tạo ra một chuẩn Bluetooth mới tiêu tốn điện năng thấp hơn 10–100 lần so với Bluetooth Classic, cho phép thiết bị chạy từ pin nhỏ trong hàng tháng đến hàng năm. BLE hoạt động ở dải tần 2.4 GHz, sử dụng 40 kênh 2 MHz và truyền dữ liệu theo từng gói ngắn.

Ngày nay BLE có mặt trên mọi điện thoại thông minh, thiết bị đeo và hầu hết SoC IoT. Chế độ **Advertising** cho phép thiết bị phát liên tục các gói beacon mà không cần kết nối đôi hướng — điện thoại xung quanh quét và nhận thông tin, ước lượng khoảng cách qua cường độ tín hiệu (RSSI). Đây là nền tảng của nhiều ứng dụng định vị trong nhà như Apple AirTag, Google Find My Network và Indoor GPS. Bù lại, BLE có ưu điểm là tiêu thụ điện cực thấp, được tích hợp rộng rãi và không cần lắp thêm hạ tầng riêng; tuy nhiên, định vị dựa trên cường độ tín hiệu chỉ đạt sai số khoảng 1–3 m và dễ bị ảnh hưởng bởi phản xạ sóng cũng như môi trường nhiều kim loại. Chính vì vậy, trong khóa luận BLE chỉ được dùng cho chức năng dẫn hướng cận cảnh ở bước cuối, chứ không đảm nhận vai trò nguồn định vị chính.

2.5 Giao thức MQTT



Hình 2.6: Logo giao thức MQTT

MQTT (Message Queuing Telemetry Transport) được phát triển bởi Andy Stanford-Clark (IBM) và Arlen Nipper (Arcom) năm 1999 để giám sát đường ống dẫn qua vệ tinh bằng thông hẹp. Giao thức được IBM mở công khai năm 2010 và trở thành chuẩn OASIS năm 2014 [8]. Ngày nay MQTT là giao thức truyền thông IoT phổ biến nhất thế giới, được tích hợp trong AWS IoT, Azure IoT Hub, Google Cloud IoT và hầu hết các nền tảng IoT thương mại.

MQTT dựa trên mô hình **Publish/Subscribe**: publisher gửi bản tin lên một Topic thông qua Broker trung gian; subscriber đăng ký Topic đó và nhận bản tin tức thời. Thiết bị không cần biết địa chỉ nhau — tất cả liên lạc qua Broker. Header gói tin tối thiểu chỉ 2 byte, kết nối TCP liên tục thay vì mở/đóng mỗi lần — phù hợp cho thiết bị tài nguyên thấp gửi dữ liệu liên tục. Tính năng **Last Will and Testament (LWT)**: thiết bị đặt trước "di chúc" với Broker; nếu mất kết nối đột ngột, Broker tự gửi thông báo đến các bên liên quan — phát hiện thiết bị ngoại tuyến mà không cần liên tục dò hỏi. Nhìn chung, MQTT có ưu điểm là nhẹ, hỗ trợ mã hóa TLS và ba mức bảo đảm gửi tin (QoS); nhược điểm là Broker đóng vai trò điểm lỗi đơn (single point of failure) và bản thân giao thức không có cơ chế tự khám phá topic. Những đặc tính này khiến MQTT trở thành lựa chọn phù hợp để truyền bản tin định vị liên tục từ các nút UWB về máy chủ trong hệ thống của khóa luận.

2.6 NestJS

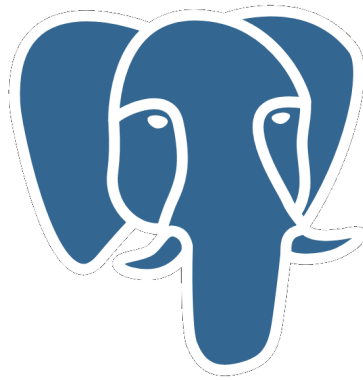


Hình 2.7: Logo NestJS

NestJS [4] được Kamil Myśliwiec phát hành năm 2017 với mục tiêu mang kiến trúc rõ ràng của Angular vào phía server Node.js. Trước NestJS, hầu hết dự án Node.js dùng Express thuần túy — linh hoạt nhưng không có quy ước kiến trúc nào, dẫn đến code khó bảo trì khi dự án mở rộng. NestJS giải quyết vấn đề này bằng kiến trúc **Module** rõ ràng: mỗi nhóm chức năng đóng gói trong một module, tách biệt hoàn toàn với các module khác. Cơ chế **Dependency Injection** tự động quản lý vòng đời và cung cấp các phụ thuộc — lập trình viên khai báo cần gì, framework tự lo khởi tạo và truyền vào.

NestJS hiện có hơn 68 nghìn sao trên GitHub (tháng 6/2025), được dùng bởi nhiều công ty lớn như Adidas, Autodesk, Roche và các startup toàn cầu. Framework hỗ trợ đồng thời nhiều giao thức giao tiếp (HTTP REST, WebSocket, MQTT, gRPC, microservices) trong cùng một nền mã — một ưu điểm lớn cho hệ thống IoT cần xử lý nhiều nguồn dữ liệu. Bù lại, NestJS khởi động chậm hơn Express thuần và có độ dốc học tập cao hơn đối với người mới bắt đầu.

2.7 Cơ sở dữ liệu PostgreSQL và pgvector



Hình 2.8: Logo PostgreSQL

PostgreSQL bắt đầu là dự án nghiên cứu POSTGRES tại UC Berkeley năm 1986 dưới sự dẫn dắt của Michael Stonebraker. Sau nhiều lần tái cấu trúc, phiên bản mã nguồn mở hoàn chỉnh ra mắt năm 1996 với tên PostgreSQL. Đây là RDBMS mã nguồn mở tuân thủ chuẩn SQL nghiêm ngặt nhất, nổi bật với hệ thống extension cho phép bổ sung kiểu dữ liệu và chức năng mới mà không cần sửa code lõi.

Theo khảo sát Stack Overflow Developer Survey 2024, PostgreSQL là cơ sở dữ liệu được lập trình viên yêu thích nhất trong nhiều năm liên tiếp. Nhiều công ty lớn như Apple, Instagram, Spotify và Cisco dùng PostgreSQL ở quy mô hàng tỷ bản ghi. Extension **pgvector** (ra mắt 2021) bổ sung khả năng lưu trữ và tìm kiếm vector nhúng (embedding) — cho phép xây dựng tìm kiếm ngữ nghĩa ngay trong database quan hệ mà không cần hệ thống vector database riêng như Pinecone hay Weaviate. Nhìn chung, PostgreSQL nổi bật ở tính toàn vẹn giao dịch (ACID), khả năng mở rộng và bộ tính năng phong phú; đổi lại, hệ quản trị này tiêu tốn tài nguyên nhiều hơn và cấu hình phức tạp hơn so với MySQL.

2.8 Redis

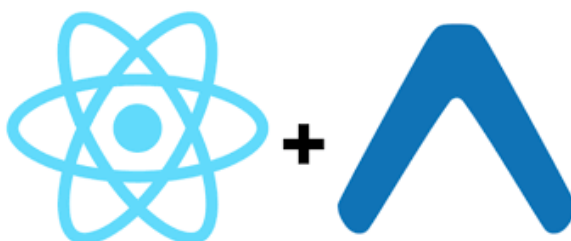


Hình 2.9: Logo Redis

Redis (Remote Dictionary Server) do Salvatore Sanfilippo phát triển năm 2009 để giải quyết bài toán đếm trang xem thời gian thực mà MySQL không đủ nhanh. Ngay từ đầu Redis chọn triết lý lưu toàn bộ dữ liệu trong RAM để đạt tốc độ tối đa, sau đó mới bổ sung tùy chọn bền vững (persistence) cho các trường hợp cần. Tốc độ đọc/ghi đạt hàng triệu thao tác mỗi giây, latency dưới 1 mili-giây — nhanh hơn 10–100 lần so với database lưu trên ổ đĩa.

Redis là in-memory database phổ biến nhất thế giới, được dùng bởi Twitter, GitHub, Snapchat, Stack Overflow và hàng triệu ứng dụng khác. Ngoài tốc độ, Redis hỗ trợ nhiều cấu trúc dữ liệu phong phú (String, Hash, List, Set, Sorted Set, Stream) và các tính năng quan trọng: TTL tự động xóa key sau thời gian định trước, Pub/Sub cho giao tiếp bất đồng bộ giữa các process. Redis phù hợp cho cache, session, hàng đợi tác vụ và lưu trạng thái thời gian thực. Hạn chế của Redis là dữ liệu có thể mất nếu máy chủ tắt đột ngột, và do lưu toàn bộ trong RAM (đắt hơn ổ đĩa) nên không phù hợp để lưu trữ khối lượng lớn lâu dài.

2.9 React Native và Expo



Hình 2.10: Logo React Native và Expo

React Native được Facebook (nay là Meta) phát triển nội bộ từ năm 2013 và mã nguồn mở vào năm 2015, xuất phát từ hackathon "React JS for native mobile" do Jordan Walke khởi xướng. Thay vì WebView như Cordova/Ionic, React Native dịch các component React trực tiếp sang widget native của từng nền tảng (UIKit/SwiftUI trên iOS, Android View trên Android) — cho hiệu năng giao diện gần với ứng dụng viết native, trong khi vẫn dùng một bộ code TypeScript chung.

React Native hiện là framework di động đa nền tảng phổ biến thứ hai sau Flutter, được dùng bởi Microsoft, Shopify, Discord, Coinbase và hàng nghìn công ty khác. Expo SDK là lớp bổ sung cung cấp API thống nhất cho camera, thông báo đẩy, lưu trữ bảo mật và OTA update (đẩy bản cập nhật JavaScript trực tiếp đến người dùng mà không cần phát hành lại qua app store). React Native phù hợp khi cần ứng dụng đa nền tảng với trải nghiệm gần với ứng dụng gốc, đội ngũ nhỏ và cần cập nhật nhanh. Tuy vậy, với đồ họa nặng, hiệu năng vẫn kém hơn ứng dụng gốc thuần, và một số tính năng hệ thống đòi hỏi viết mô-đun gốc riêng.

2.10 Server-Sent Events (SSE) và Socket.IO

Server-Sent Events (SSE) được W3C chuẩn hóa năm 2006 như một giải pháp đơn giản hơn WebSocket cho luồng dữ liệu một chiều từ server xuống client. SSE dùng kết nối HTTP thông thường được giữ mở — server gửi từng sự kiện dạng văn bản theo định dạng sự kiện SSE chuẩn, client tự động reconnect khi mất kết nối. Không cần thêm thư viện phía client; trình duyệt và nhiều HTTP client hỗ trợ sẵn qua EventSource API. Phù hợp cho luồng thông báo, cập nhật trạng thái và truyền dòng phản hồi của trợ lý AI. Ưu điểm của SSE là đơn giản, hoạt động tốt qua HTTP/2 và thân thiện với proxy; hạn chế là chỉ truyền một chiều và chỉ hỗ trợ văn bản UTF-8.

Socket.IO ra mắt năm 2010 bởi Guillermo Rauch, xây trên WebSocket nhưng bổ sung nhiều tính năng: tự động fallback về HTTP long-polling, phòng (room) cho nhóm client, namespace cho phân luồng logic, acknowledgement xác nhận nhận bản tin, và reconnect tự động với backoff. WebSocket chuẩn (RFC 6455, 2011) cũng được các framework hiện đại tích hợp trực tiếp. Socket.IO phù hợp cho giao tiếp hai chiều thời gian thực như chat, game và đồng bộ trạng thái giữa nhiều máy khách. Socket.IO có ưu điểm là giao tiếp hai chiều, hỗ trợ dữ liệu nhị phân và cơ chế dự phòng tốt; bù lại, nó nặng hơn SSE hay WebSocket thuần và cần thư viện riêng ở phía máy khách.

2.11 Kỹ thuật Hybrid RAG (Retrieval-Augmented Generation)

RAG (Retrieval-Augmented Generation) [3] do Lewis và cộng sự (2020) đề xuất nhằm khắc phục hai hạn chế của LLM thuần là kiến thức tĩnh và ảo giác. Ý

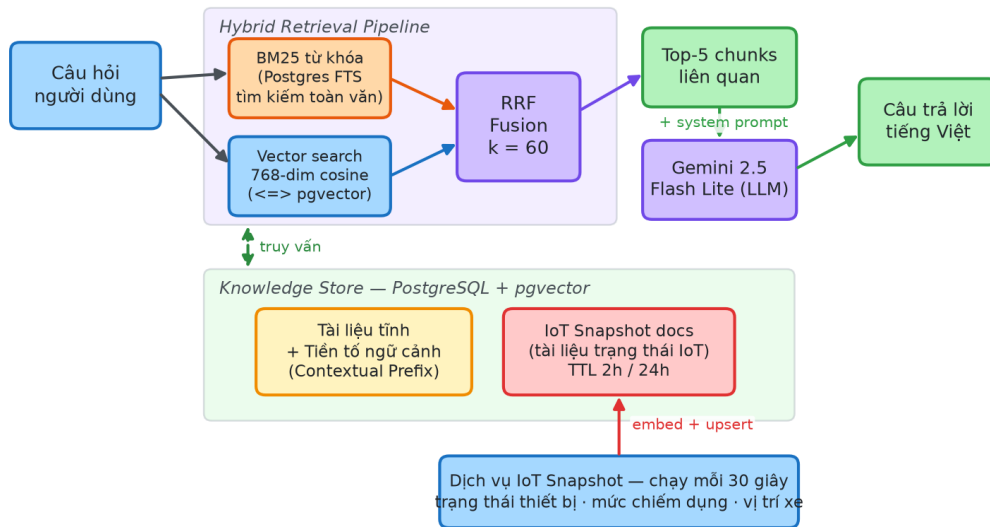
tường cốt lõi gồm ba bước: **truy xuất** (Retrieve) các tài liệu liên quan từ kho tri thức, **tăng cường** (Augment) chúng vào prompt, rồi để LLM **sinh** (Generate) câu trả lời dựa trên thông tin thực tế vừa cung cấp thay vì chỉ dựa vào kiến thức tĩnh từ lúc huấn luyện. Nhờ đó hệ thống vừa giảm bịa đặt do có dữ liệu thực làm căn cứ, vừa bổ sung được tri thức nội bộ cập nhật liên tục.



Hình 2.11: Mô hình Retrieval-Augmented Generation (RAG)

Tuy nhiên, RAG thuần vector (chỉ dùng cosine similarity giữa các embedding) bỏ sót các truy vấn chứa từ khóa kỹ thuật chính xác — ví dụ câu hỏi "Thiết bị AA03 có đang offline không?" chứa mã "AA03" vốn không mang ngữ nghĩa rõ trong không gian vector, khiến embedding khó tìm đúng đoạn chứa từ khóa này. Để khắc phục, đề tài triển khai **Hybrid RAG** kết hợp tìm kiếm từ khóa và tìm kiếm ngữ nghĩa, đồng thời tích hợp **IoT Snapshot Documents** để đưa trạng thái hệ thống thời gian thực vào ngữ cảnh. Hệ thống gồm hai thành phần: **Knowledge Store** lưu và đánh chỉ mục tài liệu, và **Retrieval Pipeline** xử lý mỗi câu hỏi qua hai kênh song song rồi hợp nhất kết quả; Hình 2.12 minh họa toàn bộ kiến trúc cùng vòng cập nhật dữ liệu IoT thời gian thực.

Kiến trúc Hybrid RAG tích hợp IoT Snapshot Documents



Hình 2.12: Kiến trúc Hybrid RAG tích hợp IoT Snapshot

Knowledge Store lưu hai loại document trong cùng một bảng trên PostgreSQL kết hợp extension **pgvector** — cho phép lưu embedding 768 chiều và tìm kiếm lân cận gần nhất theo cosine distance ngay trong cơ sở dữ liệu sẵn có, không phải vận hành thêm một dịch vụ vector riêng. Hai loại tài liệu được phân biệt qua cột `chunk_type` như ở Bảng 2.2; mỗi đoạn còn được PostgreSQL tự sinh một cột TSVECTOR (có index GIN) phục vụ tìm kiếm từ khóa.

Loại tài liệu	chunk_type	Nguồn	TTL	Ví dụ nội dung
Tài liệu tĩnh	static	Markdown mô tả hệ thống	Vĩnh viễn	"UWB EWT550 độ chính xác ±10 cm..."
Snapshot IoT	iot_snapshot	IoTSnapshotService (5 phút)	2 giờ	"[SNAPSHOT 09:32] Tầng B1: 12 slot trống"
Cảnh báo IoT	iot_snapshot	IoTSnapshotService	24 giờ	"[SNAPSHOT 09:35] Thiết

				bị AA03 offline"
--	--	--	--	---------------------

Truy hồi lai (Hybrid Search). Mỗi câu hỏi được xử lý qua hai kênh song song. Kênh **BM25** [10] dùng Full-Text Search của PostgreSQL, xếp hạng theo tần suất và tầm quan trọng của từ khóa — hiệu quả với thuật ngữ kỹ thuật như "AA03", "EWT550", "tầng B1". Kênh **Vector** chuyển câu hỏi thành vector 768 chiều rồi lấy các tài liệu gần nhất theo cosine distance — hiệu quả với truy vấn ngữ nghĩa như "xe đang đỗ ở đâu", "tầng nào còn chỗ". Hai danh sách được hợp nhất bằng **Reciprocal Rank Fusion (RRF)** với hằng số $k = 60$ (giá trị chuẩn theo Cormack và cộng sự [9]):

$$\text{RRF}(d) = \sum_r \frac{1}{60 + r(d)}$$

trong đó $r(d)$ là thứ hạng của tài liệu d trong mỗi danh sách kết quả. Tài liệu xuất hiện ở thứ hạng cao trong cả hai kênh nhận điểm RRF cao nhất; toàn bộ được thực hiện bằng một câu lệnh SQL lấy top-5, kèm bộ lọc thời gian để loại các snapshot IoT đã cũ.

IoT Snapshot Documents — đóng góp chính ở lớp AI. Thay vì chỉ lưu tài liệu tĩnh, hệ thống liên tục sinh các "knowledge document" mô tả trạng thái thực tế của garage rồi upsert vào cùng kho vector. Dịch vụ IoTSnapshotService chạy mỗi 5 phút và tạo ba loại snapshot: (1) trạng thái thiết bị (so sánh danh sách online trong Redis với toàn bộ board), ví dụ *[SNAPSHOT 09:32] Online: AA01, AA02, AA04, AB00; Offline: AA03*; (2) mức lấp đầy từng tầng, ví dụ *[SNAPSHOT 09:32] Tầng B1: đã đỗ 18, còn trống 12*; (3) vị trí xe gần nhất, ví dụ *Xe AB00 ở tầng B1, tọa độ (12,35 m, 8,52 m)*. Khi người dùng hỏi "Tầng B1 còn bao nhiêu chỗ?", pipeline tìm đúng snapshot nhờ cả từ khóa lẫn ngữ nghĩa, giúp trợ lý trả lời theo dữ liệu thực; một job cleanup định kỳ xóa snapshot cũ (thường 2 giờ, cảnh báo 24 giờ) để dữ liệu luôn phản ánh hiện trạng. Ngoài ra, mỗi tài liệu tĩnh được thêm một dòng prefix ngữ cảnh trước khi embed (kỹ thuật Contextual Retrieval [14]) nhằm tăng tỉ lệ truy hồi đúng.

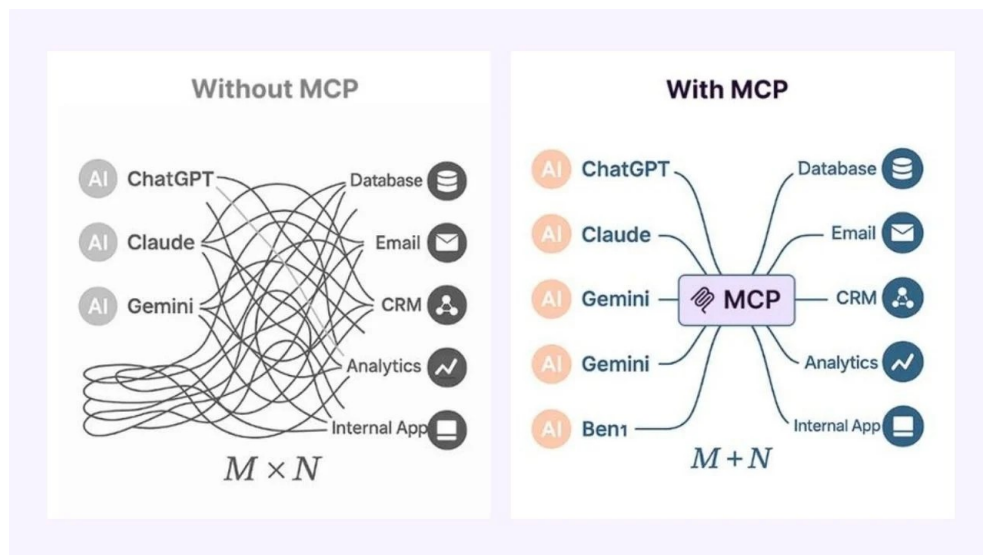
Trong vận hành, hệ thống lấy 5 chunk liên quan nhất nối vào prompt trước khi gọi LLM; nếu cần dữ liệu cụ thể hơn, LLM gọi MCP tool tương ứng (§2.12). Hai lớp bổ sung nhau: RAG cung cấp ngữ cảnh tổng quan, còn MCP tool cung cấp dữ liệu

chính xác theo yêu cầu. Bảng 2.3 tóm tắt ba cấu hình RAG được thiết kế để thực nghiệm so sánh ở Chương 4; trong đó Advanced-B là bản triển khai cuối, còn Baseline được giữ lại để đối chứng. Kết quả đo (chi tiết tại §4.3.6) xác nhận vai trò của truy hồi ngữ nghĩa: trên bộ 13 câu hỏi gán nhãn, cấu hình Hybrid đạt **Recall@5 = 0,821** và **MRR = 0,962**, trong khi BM25 thuần chỉ đạt 0,026.

Cấu hình	BM25	Vector Search	IoT Snapshot	Contextual Prefix
Baseline (vanilla RAG)	Không	Có	Không	Không
Advanced-A (Hybrid RAG)	Có	Có	Có	Không
Advanced-B (Hybrid + Contextual)	Có	Có	Có	Có

2.12 Giao thức MCP (Model Context Protocol)

MCP (Model Context Protocol) [5] được Anthropic công bố cuối năm 2024 như một chuẩn mở cho phép LLM gọi các công cụ bên ngoài theo cách nhất quán. Trước MCP, mỗi hệ thống AI tích hợp công cụ theo cách riêng; MCP chuẩn hóa giao thức: LLM sinh yêu cầu gọi công cụ dạng JSON có cấu trúc → MCP Server thực thi → trả kết quả về. Công cụ và LLM tách biệt hoàn toàn — cùng một bộ công cụ có thể dùng với bất kỳ LLM nào hỗ trợ MCP. MCP nhanh chóng được hỗ trợ bởi nhiều nền tảng AI lớn (Anthropic, OpenAI, Google) và nhiều môi trường lập trình. Ưu điểm của MCP là chuẩn hóa, tái sử dụng và dễ mở rộng; nhược điểm là thêm một chặng trung gian làm tăng nhẹ độ trễ, và bản thân chuẩn còn mới nên vẫn đang tiếp tục phát triển.



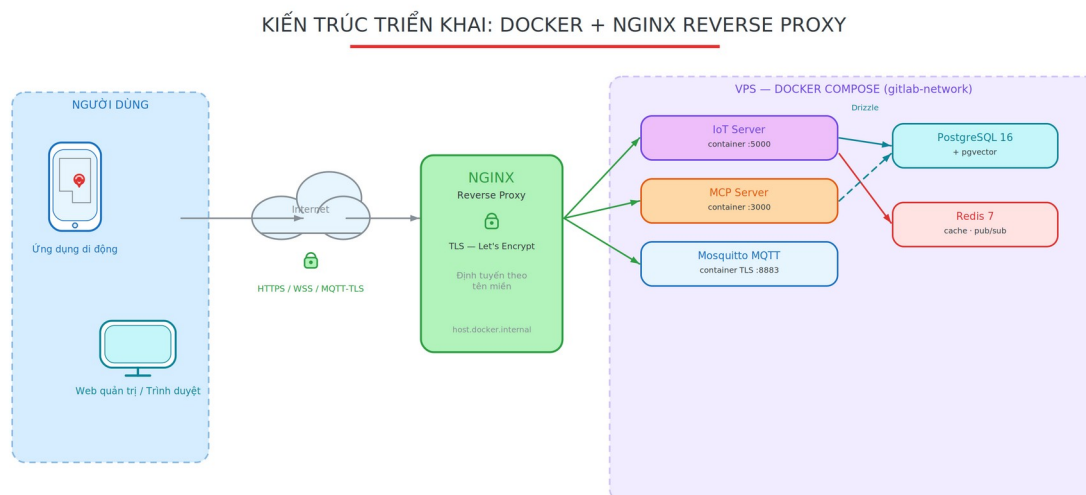
Hình 2.13: Model Context Protocol (MCP)

2.13 Docker và Nginx

Docker được Solomon Hykes giới thiệu tại PyCon 2013, mã nguồn mở ngay năm đó. Docker giải quyết vấn đề kinh điển "chạy được trên máy tôi nhưng lỗi trên server" bằng cách đóng gói ứng dụng cùng toàn bộ runtime, thư viện và cấu hình vào một container độc lập. Container chạy nhất quán trên mọi máy có Docker engine, không phụ thuộc hệ điều hành host. Docker thay đổi hoàn toàn cách triển khai phần mềm: theo khảo sát Stack Overflow 2024, Docker là công cụ DevOps được dùng nhiều nhất toàn cầu. **Docker Compose** cho phép định nghĩa hệ thống đa service trong một file YAML và khởi động tất cả bằng một lệnh — lý tưởng cho môi trường phát triển và triển khai trên VPS. Ưu điểm của Docker là bảo đảm môi trường nhất quán, dễ mở rộng quy mô và cô lập từng dịch vụ; hạn chế là các container dùng chung nhân (kernel) của máy chủ nên mức cô lập bảo mật thấp hơn máy ảo.

Nginx (phát âm "engine-x") được Igor Sysoev phát triển từ 2002 để giải quyết vấn đề C10K — xử lý 10.000 kết nối đồng thời trên một server, điều Apache httpd không làm được lúc bấy giờ. Nginx dùng kiến trúc event-driven bất đồng bộ: một process có thể xử lý hàng chục nghìn kết nối mà không cần mỗi kết nối một thread riêng. Nginx hiện phục vụ khoảng 30–40% trong số 1 tỷ website hàng đầu thế giới. Vai trò reverse proxy của Nginx trong hệ thống phân tán: đứng trước nhiều backend service, nhận request từ internet, định tuyến dựa trên tên miền hay đường dẫn, xử lý TLS, load balancing và cache tĩnh. Ưu điểm của Nginx là nhẹ, thông lượng cao và

cấu hình linh hoạt; nhược điểm là cú pháp cấu hình có độ dốc học tập nhất định, và một số mô-đun phải biên dịch từ mã nguồn.



Hình 2.14: Kiến trúc triển khai Docker + Nginx

2.14 Mô hình ngôn ngữ lớn (LLM) và nền tảng chatbot

Mô hình ngôn ngữ lớn (Large Language Model — LLM) là nền tảng của lớp trợ lý hỏi đáp trong đề tài. Quá trình phát triển của LLM có thể tóm lược qua sáu giai đoạn:

- **Trước 2013 — n-gram:** ước lượng xác suất từ tiếp theo từ vài từ trước, của số ngữ cảnh ngắn.
- **2013–2017 — RNN/LSTM:** xử lý chuỗi tuần tự, nhớ phụ thuộc xa nhưng khó song song hóa.
- **2017 — Transformer:** cơ chế tự chú ý (self-attention) cho phép song song hóa, là nền của mọi LLM hiện đại.
- **2018–2020 — Tiền huấn luyện (BERT, GPT):** học trên corpus khổng lồ, GPT-3 thể hiện khả năng few-shot.
- **2021–2022 — Scaling và RLHF:** mở rộng quy mô và tinh chỉnh theo phản hồi con người, sinh ra ChatGPT.
- **2023–nay — RAG, agentic, đa phương thức:** gắn truy hồi tri thức và gọi công cụ, xử lý cả văn bản, hình ảnh, âm thanh.

Dù mạnh, LLM vẫn mang hai hạn chế căn bản là **kiến thức tĩnh** (không biết dữ liệu nội bộ hay sự kiện mới sau huấn luyện) và **ảo giác** (sinh văn bản mạch lạc nhưng sai sự thật). Hai kỹ thuật trực tiếp khắc phục các hạn chế này — Hybrid RAG (§2.11) và MCP tool-calling (§2.12) — chính là cơ sở để xây dựng chatbot của đề tài. Về vận hành, chatbot dùng cơ chế **dự phòng nhà cung cấp tự động**: mô hình chính là Gemini 2.5 Flash Lite, khi gặp lỗi vượt hạn mức (429) hoặc lỗi máy chủ (5xx) sẽ tự chuyển sang DeepSeek mà người dùng không cảm nhận gián đoạn.

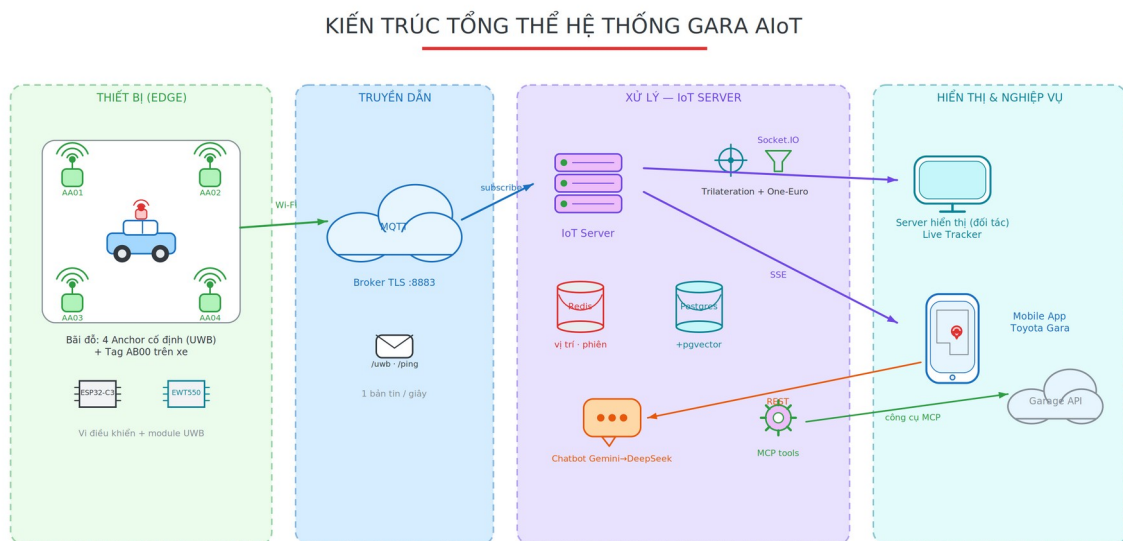
Kết luận chương 2. Chương 2 đã trình bày cơ sở lý thuyết của các công nghệ cốt lõi được sử dụng trong hệ thống: lớp định vị (UWB, Two-Way Ranging, Trilateration, bộ lọc One-Euro), lớp phần cứng (ESP32-C3, BLE), lớp truyền thông (MQTT, SSE/Socket.IO), lớp xử lý và lưu trữ (NestJS, PostgreSQL/pgvector, Redis), lớp giao diện (React Native/Expo), lớp trí tuệ nhân tạo (RAG, MCP, lịch sử phát triển LLM) và hạ tầng triển khai (Docker, Nginx). Mỗi công nghệ đều được nêu rõ nguyên lý, ưu – nhược điểm và lý do lựa chọn, làm nền tảng để đề xuất kiến trúc và phương pháp ở Chương 3.

CHƯƠNG 3: PHƯƠNG PHÁP ĐỀ XUẤT

Trên nền các cơ sở lý thuyết đã trình bày ở Chương 2, chương này đề xuất kiến trúc và phương pháp hiện thực hóa hệ thống. Nội dung lần lượt mô tả kiến trúc tổng thể, thiết kế phần cứng và firmware của các nút UWB, chuỗi xử lý định vị tại máy chủ, cơ chế trợ lý hỏi đáp, quy trình vận hành một lượt đỗ xe từ lúc vào đến lúc ra, và sau cùng là mô hình tích hợp với hệ thống quản lý gara của bên thứ ba.

3.1 Mô tả tổng quan hệ thống đề xuất

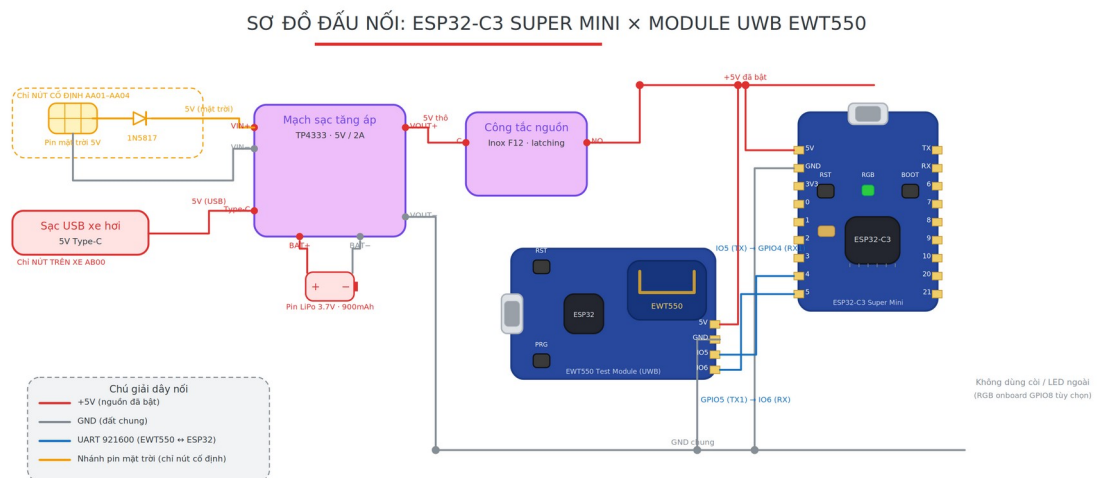
Để giải quyết bài toán giám sát và quản lý bãi đỗ xe trong nhà, khóa luận đề xuất một kiến trúc AIoT hoàn chỉnh được tổ chức thành bốn tầng liên kết chặt chẽ, trải dài từ thiết bị vật lý đến giao diện người dùng. Tầng dưới cùng là phần cứng định vị gồm các nút UWB thu thập dữ liệu khoảng cách; kế đến là tầng truyền tin dùng giao thức MQTT để chuyển bản tin về máy chủ; tầng máy chủ (IoT Server) đảm nhận toàn bộ phần xử lý cốt lõi gồm giải tọa độ, lọc nhiễu và phân phối dữ liệu thời gian thực; trên cùng là tầng ứng dụng gồm ứng dụng di động cho người dùng cuối và giao diện quản trị web. Trọng tâm của kiến trúc nằm ở tầng máy chủ — nơi biến dữ liệu khoảng cách thô từ UWB thành tọa độ phương tiện chính xác và ổn định. Bên cạnh lớp định vị nền tảng đó, hệ thống tích hợp thêm một lớp trợ lý hỏi đáp ứng dụng kỹ thuật Hybrid RAG, đóng vai trò tiện ích hỗ trợ giúp người vận hành tra cứu dữ liệu bằng ngôn ngữ tự nhiên. Kiến trúc tổng thể được minh họa trong Hình 3.1.



Hình 3.1: Kiến trúc tổng thể hệ thống AIoT

3.2 Thiết kế phần cứng và Firmware

Mỗi nút định vị trong hệ thống được xây dựng quanh vi điều khiển ESP32-C3 Super Mini ghép với module UWB EWT550 qua giao tiếp UART. Cùng một thiết kế phần cứng được dùng cho hai vai trò khác nhau tùy theo cấu hình firmware: các Anchor được lắp cố định tại bốn góc của mỗi tầng để làm mốc tọa độ, còn Tag được gắn trên phương tiện để xác định vị trí. Trong mô hình hoạt động, Tag trên xe đóng vai trò nút chủ động đo khoảng cách Two-Way Ranging (TWR) tới cả bốn Anchor mỗi giây, đóng gói toàn bộ kết quả vào một bản tin duy nhất rồi gửi về máy chủ qua giao thức MQTT có mã hóa TLS. Việc gộp khoảng cách tới mọi Anchor trong một bản tin cho phép máy chủ định vị độc lập theo từng gói mà không phải chờ ghép dữ liệu từ nhiều nguồn rời rạc. Sơ đồ đấu nối phần cứng của một nút được trình bày trong Hình 3.2.

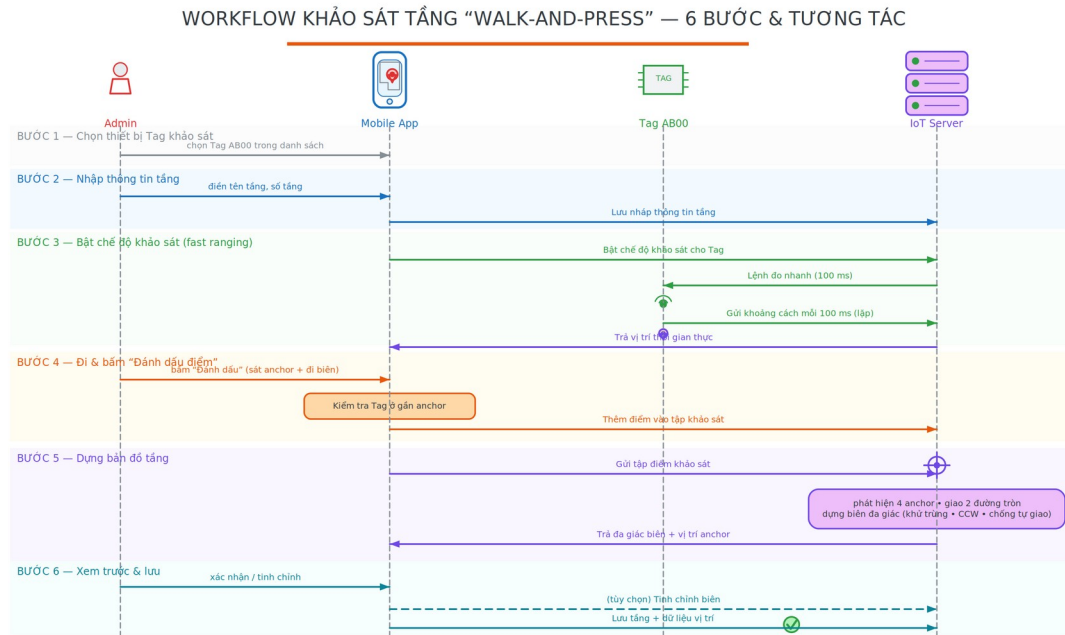


Hình 3.2: Sơ đồ đấu nối mạch phần cứng nút UWB



Hình 3.3: Một nút UWB thực tế đã lắp ráp (ESP32-C3 Super Mini ghép module UWB EWT550) — dùng chung cho cả Anchor và Tag, chỉ khác cấu hình firmware

Trước khi đưa vào vận hành, mỗi tầng hầm được lập bản đồ bằng phương pháp khảo sát **walk-and-press**: kỹ thuật viên mang Tag đi tới sát từng Anchor rồi đi dọc ranh giới bãi, nhấn nút đánh dấu tại mỗi điểm. IoT Server tự động phát hiện bốn mẫu đo đứng sát Anchor (khoảng cách nhỏ nhất dưới 50 cm), suy ra khoảng cách chéo giữa các Anchor để dựng hệ tọa độ tầng, sau đó Trilateration các điểm biên thành đa giác khép kín mô tả ranh giới bãi đỗ. Để giảm số đỉnh thừa do đo đặc dày đặc, hệ thống áp dụng thuật toán đơn giản hóa đường đa giác Douglas-Peucker [11] (ngưỡng $\epsilon = 0,3$ m) trước khi lưu vào cơ sở dữ liệu.

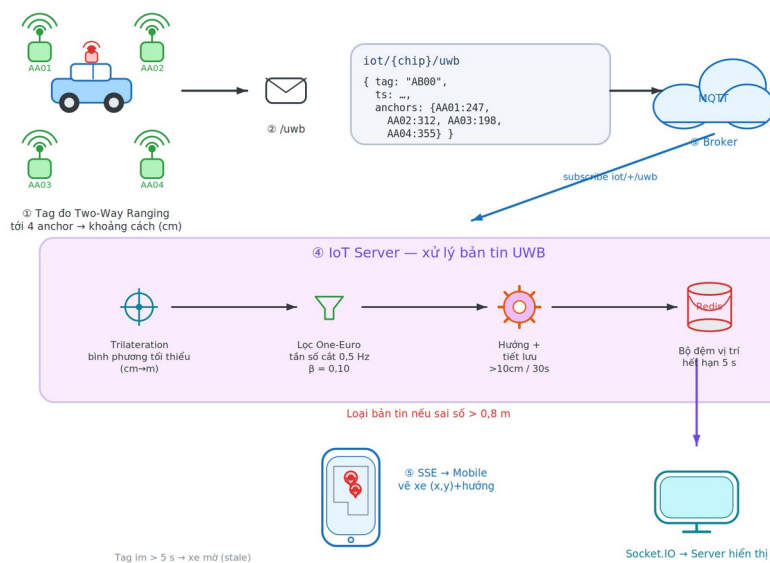


Hình 3.4: Quy trình khảo sát bãi đỗ (walk-and-press)

3.3 Quy trình xử lý tại IoT Server

IoT Server được xây dựng trên nền NestJS và giữ vai trò trung tâm xử lý của toàn hệ thống, nơi mọi dữ liệu định vị thô được biến đổi thành thông tin vị trí có thể sử dụng. Khi một bản tin khoảng cách từ Tag được gửi đến qua MQTT, máy chủ lần lượt thực hiện một chuỗi bốn bước gắn kết. Trước hết, máy chủ tiếp nhận và bóc tách bản tin để lấy ra khoảng cách từ Tag tới từng Anchor. Tiếp đó, dựa trên tọa độ đã biết của các Anchor, máy chủ áp dụng thuật toán Trilateration theo phương pháp bình phương tối thiểu để giải ra tọa độ (x, y) của phương tiện. Vì tín hiệu vô tuyến luôn kèm theo nhiễu, tọa độ vừa giải được tiếp tục đi qua bộ lọc One-Euro nhằm làm trơn quỹ đạo và loại bỏ những bước nhảy đột ngột do hiện tượng đa đường (multipath) gây ra. Cuối cùng, kết quả đã được làm sạch được ghi vào bộ nhớ đệm Redis và đẩy tới ứng dụng di động qua cơ chế Server-Sent Events (SSE), bảo đảm người dùng nhìn thấy vị trí xe gần như tức thời. Toàn bộ chuỗi xử lý này được minh họa trong Hình 3.5; chi tiết các tham số thiết kế của từng khâu được trình bày sâu hơn ở Mục 4.5.

LUỒNG DỮ LIỆU ĐỊNH VỊ UWB THỜI GIAN THỰC



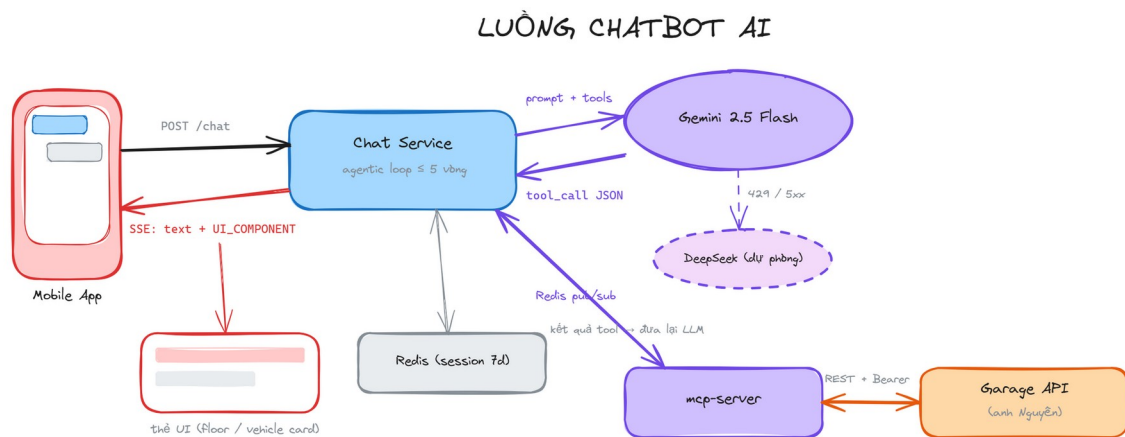
Hình 3.5: Luồng dữ liệu định vị UWB

Ngoài việc phân phối tọa độ tức thời, IoT Server còn lưu vết lịch sử vị trí của mỗi phương tiện vào một bảng dữ liệu chuỗi thời gian nhằm phục vụ phân tích vận hành về sau. Trên cơ sở dữ liệu này, hệ thống cung cấp ba phép phân tích: (i) **thời gian lưu bãi** (dwell-time) của từng xe — tổng thời gian hiện diện, tách theo phiên đỗ; (ii) **bản đồ nhiệt mức độ sử dụng** (occupancy heatmap) — lưới ô phủ mặt bằng, mỗi ô tích lũy thời gian phương tiện lưu lại; và (iii) **lưu lượng đỉnh** (peak occupancy) — số phương tiện đồng thời theo từng khung thời gian. Các phép phân tích được hiện thực dưới dạng hàm thuần, tách khỏi tầng truy xuất dữ liệu nên kiểm thử được độc lập, và được lộ ra qua REST để giao diện quản trị và chatbot khai thác (trình bày chi tiết ở Mục 4.7).

3.4 Cơ chế Hybrid RAG cho Gara thông minh

Bên cạnh lớp định vị nền tảng, hệ thống tích hợp một trợ lý hỏi đáp như một tiện ích bổ trợ ở lớp trí tuệ nhân tạo, giúp người vận hành tra cứu trạng thái bãi xe bằng ngôn ngữ tự nhiên. Thách thức đặt ra là trạng thái bãi xe thay đổi liên tục, trong khi kỹ thuật Truy hồi – Tăng cường Sinh (Retrieval-Augmented Generation – RAG) kinh điển vốn được thiết kế cho kho tài liệu tĩnh. Để dung hòa, hệ thống thực hiện chụp ảnh trạng thái (snapshot) dữ liệu định vị và trạng thái các ô đỗ mỗi 5 phút. Chu

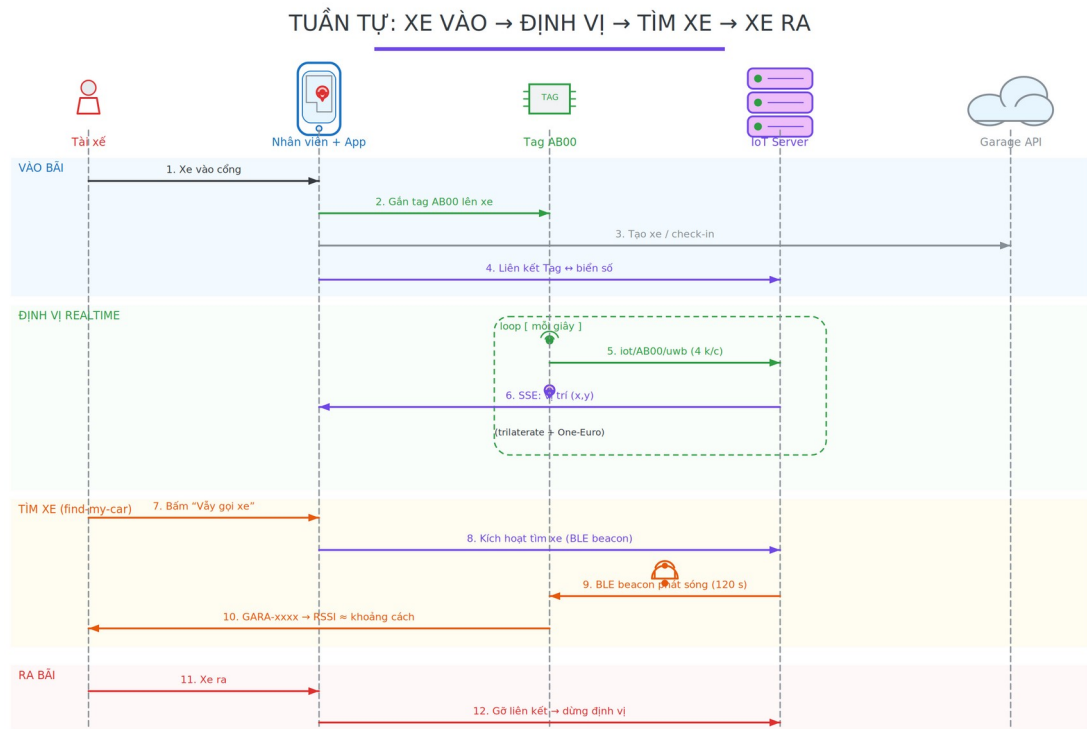
kỳ này được chọn để cân bằng giữa độ tươi của ngữ cảnh và giới hạn quota embedding (288 lần/ngày, dưới hạn mức miễn phí 1.000 lần/ngày); hệ quả là độ trễ cập nhật ngữ cảnh tối đa khoảng 5 phút. Dữ liệu này được vector hóa và lưu trữ vào PostgreSQL. Khi người dùng đặt câu hỏi, hệ thống thực hiện tìm kiếm Hybrid (Vector + Full-text) để lấy các ngữ cảnh (context) mới nhất, giúp chatbot trả lời chính xác các câu hỏi như "Còn bao nhiêu chỗ trống?" hoặc "Xe AB-123 đang ở đâu?".



Hình 3.6: Luồng Chatbot AI

3.5 Quy trình hoạt động: Xe vào — Xe ra

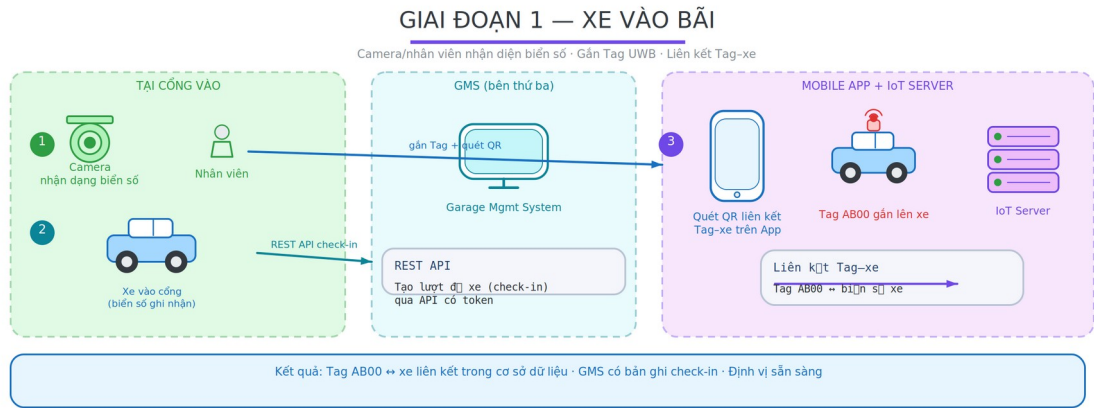
Toàn bộ vòng đời của một lượt đỗ xe — từ khi phương tiện vào đến khi rời khỏi bãi — diễn ra qua năm giai đoạn liên kết chặt chẽ giữa phần cứng IoT, IoT Server và hệ thống quản lý gara (bên thứ ba). Lược đồ tuần tự tổng quan của cả năm giai đoạn được trình bày trong Hình 3.7; sau đó từng giai đoạn được mô tả chi tiết kèm một sơ đồ minh họa riêng.



Hình 3.7: Lược đồ tuần tự tổng quan xe vào – xe ra

3.5.1 Giai đoạn 1: Xe vào bãi

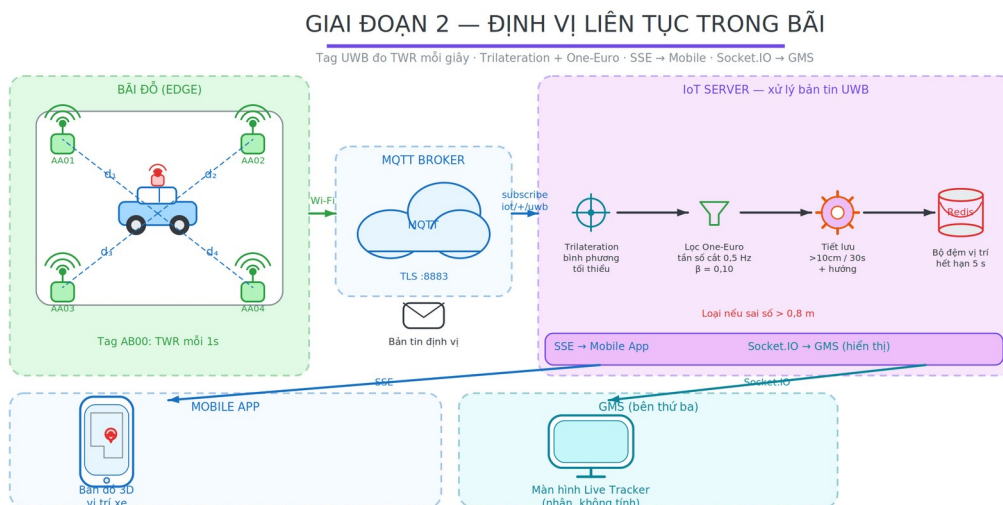
Khi phương tiện vào cổng, nhân viên hoặc hệ thống camera nhận dạng biển số ghi nhận thông tin và tạo bản ghi check-in trên **Hệ thống quản lý gara** (Garage Management System — GMS, bên thứ ba) qua REST API. Đồng thời, nhân viên gắn Tag UWB lên phương tiện và quét mã QR liên kết Tag với xe trên ứng dụng di động. Ứng dụng gọi API liên kết Tag–xe trên IoT Server để thiết lập liên kết Tag–xe trong cơ sở dữ liệu IoT. Trình tự thao tác và luồng dữ liệu của giai đoạn này được minh họa trong Hình 3.8.



Hình 3.8: Giai đoạn 1 — Xe vào bãi và liên kết Tag

3.5.2 Giai đoạn 2: Định vị liên tục trong bãi

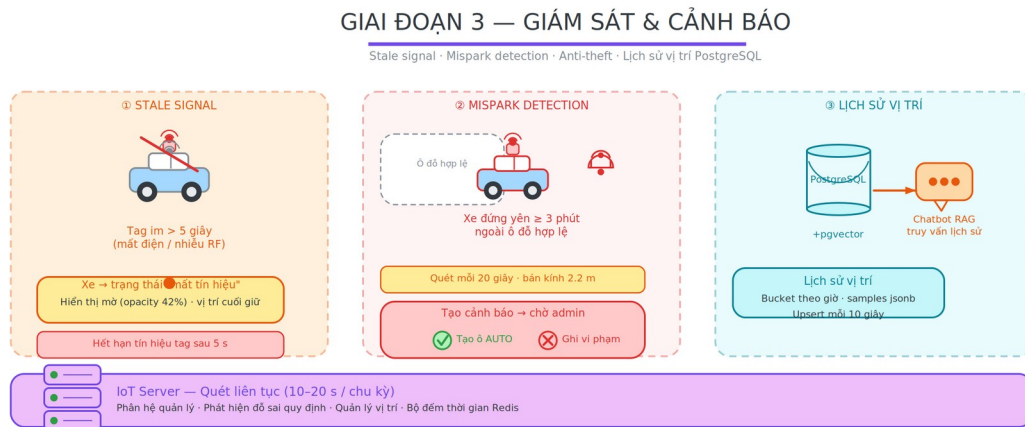
Sau khi xe được đỗ vào ô, Tag UWB trên xe bắt đầu đo khoảng cách đến bốn Anchor cố định mỗi giây và gửi kết quả qua MQTT (bản tin định vị gửi qua MQTT). IoT Server nhận gói tin, thực hiện Trilateration tính tọa độ (x, y), lọc qua One-Euro Filter rồi ghi kết quả vào Redis. Tọa độ được đẩy xuống ứng dụng di động qua SSE với mục tiêu thiết kế dưới 200 ms kể từ lúc Tag gửi tín hiệu (chi tiết đường ống ở Mục 4.5; phương pháp kiểm chứng độ chính xác ở Mục 4.6). Đồng thời, IoT Server chuyển tiếp dữ liệu vị trí **đã xử lý** đến GMS qua Socket.IO để GMS hiển thị trên màn hình quản lý của mình — GMS chỉ nhận dữ liệu, không tự tính toán định vị. Toàn bộ đường ống xử lý của giai đoạn này được minh họa trong Hình 3.9.



Hình 3.9: Giai đoạn 2 — Định vị liên tục trong bãi

3.5.3 Giai đoạn 3: Giám sát và cảnh báo

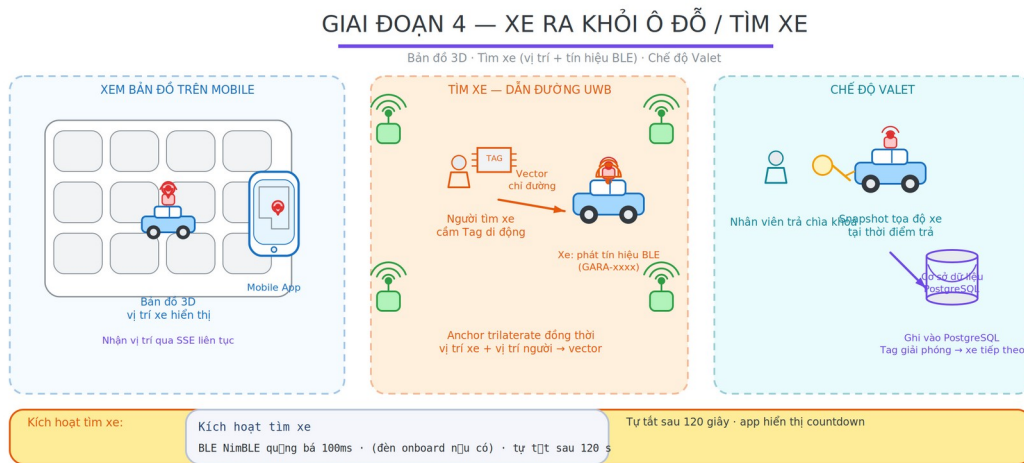
Trong suốt thời gian xe đỗ, hệ thống thực hiện song song ba nhiệm vụ giám sát. Thứ nhất là phát hiện mất tín hiệu (stale signal detection): nếu Tag không gửi dữ liệu hơn 5 giây do mất điện hoặc nhiễu RF, xe được chuyển sang trạng thái "mất tín hiệu" và hiển thị mờ trên bản đồ, trong khi IoT Server vẫn trả về vị trí cuối cùng đã biết trong snapshot REST. Thứ hai là phát hiện đỗ sai quy định (mispark detection): cứ mỗi 20 giây hệ thống quét một lượt, nếu phát hiện một xe đứng yên từ 3 phút trở lên ngoài phạm vi các ô đỗ hợp lệ thì sinh cảnh báo chờ quản trị viên xác nhận. Thứ ba là lưu vết lịch sử: IoT Server ghi hành trình của xe vào PostgreSQL theo bucket một giờ nhằm phục vụ báo cáo và trợ lý hỏi đáp. Ba cơ chế giám sát này được minh họa trong Hình 3.10.



Hình 3.10: Giai đoạn 3 — Giám sát và cảnh báo

3.5.4 Giai đoạn 4: Xe ra khỏi ô đỗ

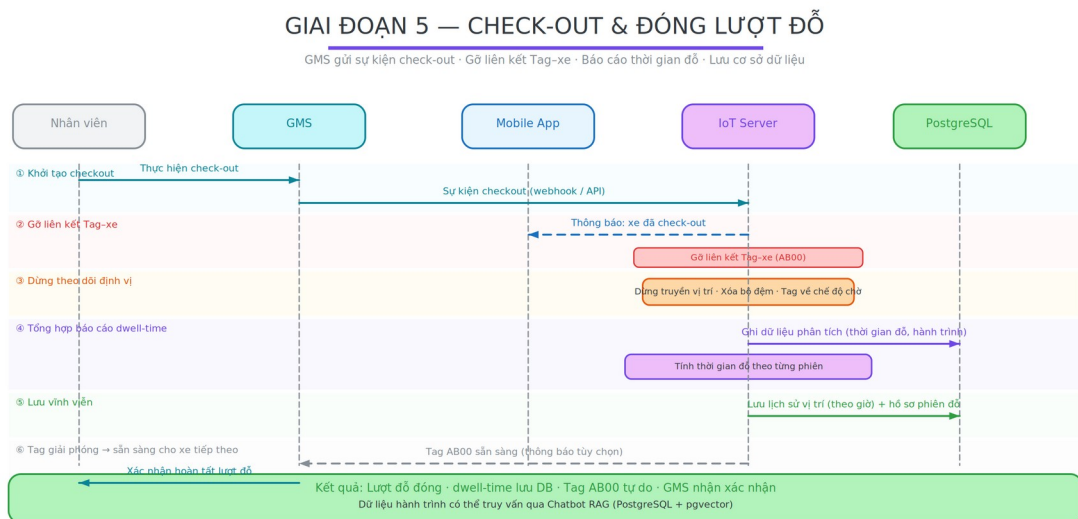
Khi chủ xe cần lấy xe, vị trí phương tiện được hiển thị trực tiếp trên bản đồ isometric 3D trong ứng dụng di động để định hướng từ xa. Để dẫn đường ở cự ly gần hơn, đề tài đề xuất giải pháp người tìm xe cầm theo một Tag UWB di động: hạ tầng Anchor sẵn có trilaterate đồng thời vị trí xe và vị trí người tìm, tính vector chỉ đường và hiển thị trên bản đồ (trình bày chi tiết tại Mục 4.2.3). Trong chế độ Valet, khi nhân viên giao trả chìa khoá, hệ thống snapshot tọa độ xe tại thời điểm đó, ghi vào dữ liệu liên kết nhân viên–xe và giải phóng Tag để nhận xe tiếp theo. Các phương án tìm xe và dẫn đường của giai đoạn này được minh họa trong Hình 3.11.



Hình 3.11: Giai đoạn 4 — Xe ra khỏi ô đồ và tìm xe

3.5.5 Giai đoạn 5: Check-out và đóng lượt đỗ

Khi xe rời khỏi bãi, nhân viên thực hiện check-out trên GMS; GMS gửi sự kiện checkout qua API. IoT Server nhận thông báo, gỡ liên kết Tag-xe (API gỡ liên kết Tag-xe), dừng theo dõi định vị và tổng hợp báo cáo dwell time cho lượt đỗ vừa kết thúc. Dữ liệu hành trình được lưu vĩnh viễn vào PostgreSQL phục vụ phân tích và truy vấn qua chatbot. Trình tự đóng lượt đỗ được minh họa trong Hình 3.12.



Hình 3.12: Giai đoạn 5 — Check-out và đóng lượt đỗ

3.6 Tích hợp Hệ thống Quản lý Gara (Bên thứ ba)

IoT Server tích hợp với GMS theo mô hình **bất đối xứng**: IoT Server đóng vai trò cung cấp dữ liệu định vị đã xử lý; GMS đóng vai trò cung cấp dữ liệu nghiệp vụ (xe, hóa đơn, nhân viên). Hai hệ thống không phụ thuộc vào nhau về mặt vận hành — IoT Server hoạt động độc lập ngay cả khi GMS tạm ngừng.

Chiều tích hợp	Giao thức	Dữ liệu
GMS → IoT Server	REST API (Bearer token)	Thông tin xe, tăng giảm, nhân viên
IoT Server → GMS	Socket.IO (client mode)	Tọa độ (x, y) đã xử lý theo thời gian thực
Mobile App → GMS	REST API	Check-in xe, tạo liên kết Tag–xe
Mobile App → IoT Server	REST + SSE	Vị trí xe, lệnh điều khiển Tag

Thiết kế này đảm bảo IoT Server có thể tích hợp với **bất kỳ GMS nào** chỉ bằng cách thay đổi endpoint URL và token xác thực — không cần thay đổi logic xử lý định vị. MCP Server gọi REST API của GMS để lấy dữ liệu nghiệp vụ phục vụ chatbot, sử dụng Bearer token riêng biệt và không chia sẻ session với IoT Server.

Kết luận chương 3. Chương 3 đã đề xuất kiến trúc AIoT tổng thể và làm rõ từng thành phần: thiết kế phần cứng – firmware UWB, quy trình khảo sát mặt bằng walk-and-press, chuỗi xử lý định vị tại IoT Server (Trilateration, bộ lọc One-Euro, phân phối thời gian thực), cơ chế Hybrid RAG cho chatbot, quy trình vận hành xe vào – xe ra và mô hình tích hợp bất đối xứng với hệ thống quản lý gara bên thứ ba. Kết quả hiện thực hóa các thiết kế này được trình bày trong Chương 4.

CHƯƠNG 4: KẾT QUẢ THỰC HIỆN

Chương này trình bày kết quả triển khai thực tế của hệ thống đã thiết kế ở Chương 3. Nội dung lần lượt mô tả môi trường triển khai, các nhóm tính năng đã hiện thực hóa trên ứng dụng di động, chatbot AI và giao diện quản trị web, sau đó trình bày phương pháp đánh giá độ chính xác định vị UWB làm cơ sở kiểm chứng chất lượng hệ thống.

4.1 Môi trường triển khai

Hệ thống được triển khai trên hai môi trường song song: môi trường phát triển (máy tính lập trình viên, WSL2/Linux) và môi trường sản xuất (VPS Linux). Phần cứng IoT gồm vi điều khiển ESP32-C3 kết nối module UWB EWT550 qua UART1. Hạ tầng backend chạy trên VPS qua Docker Compose gồm các container: IoT Server (NestJS, cổng 5000), MCP Server (Node.js, cổng 3000), Mosquitto MQTT Broker (TLS 8883), PostgreSQL 16 với pgvector, Redis 7. Nginx đảm nhận vai trò reverse proxy với HTTPS qua Let's Encrypt. Ứng dụng di động được build bằng Expo EAS và phân phối qua Firebase App Distribution.

Thành phần	Công nghệ	Môi trường
IoT Server	NestJS, Node.js 20	Docker / VPS
MCP Server	Node.js 20	Docker / VPS
MQTT Broker	Mosquitto 2 (TLS)	Docker / VPS
Database	PostgreSQL 16 + pgvector	Docker / VPS
Cache / Pub-Sub	Redis 7	Docker / VPS
Reverse proxy	Nginx + Let's Encrypt	VPS host
Mobile App	Expo SDK 54 / RN 0.81	Android APK
Firmware	ESP-IDF, ESP32-C3	Phần cứng thực

4.2 Tính năng ứng dụng di động

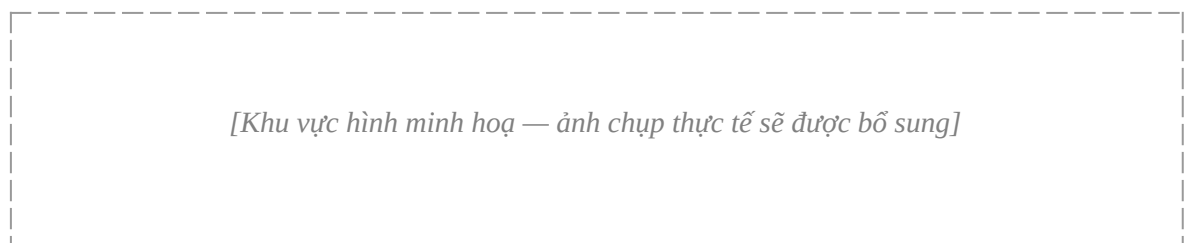
Ứng dụng di động là thành phần giao tiếp trực tiếp với người dùng cuối, có nhiệm vụ trực quan hóa dữ liệu định vị thời gian thực và cung cấp các thao tác quản lý phương tiện. Mục này lần lượt trình bày các nhóm chức năng chính của ứng dụng, từ hiển thị bản đồ, quản lý phương tiện, tìm xe, khảo sát mặt bằng đến các cơ chế giám sát và thông báo.

4.2.1 Bản đồ bãi đỗ xe 3D Isometric thời gian thực

Màn hình chính hiển thị bản đồ bãi đỗ xe theo phối cảnh isometric 3D, được dựng bằng thư viện React Native Skia với tính toán biến đổi hình học thực thi trên luồng GPU nhằm bảo đảm tốc độ khung hình ổn định. Mỗi phương tiện đang đỗ được biểu diễn bằng mô hình low-poly 3D có hướng xoay tự động theo chiều dài ô đỗ; tọa độ phương tiện được cập nhật liên tục qua Server-Sent Events (SSE) mà không cần tải lại màn hình. Người dùng có thể tương tác với bản đồ thông qua các thao tác:

- Pinch zoom (1×–4×), pan một ngón tay, double-tap phóng to về điểm chạm
- Tap vào xe để mở panel thông tin chi tiết (biển số, tọa độ X/Y, thời gian cập nhật, độ chính xác)
- Xe mất tín hiệu UWB hơn 5 giây hiển thị với độ mờ 42% và chấm màu vàng
- Điều hướng lên/xuống tầng qua danh sách hợp nhất (tầng Garage + tầng IoT)

Giao diện bản đồ định vị thời gian thực trên ứng dụng được minh họa trong Hình 4.1.



Hình 4.1: Giao diện bản đồ định vị 3D trên ứng dụng

4.2.2 Quản lý xe và QR Code

Màn hình danh sách phương tiện hiển thị toàn bộ xe đang đỗ kèm các thuộc tính biển số, loại xe và thời điểm vào bãi. Chức năng thêm phương tiện mới hỗ trợ

hai luồng nhập liệu: nhập thủ công và quét mã QR bằng camera nhằm giảm sai sót và rút ngắn thời gian thao tác. Mỗi phương tiện có gắn thiết bị định vị được liên kết với một Tag UWB thông qua màn hình chọn thiết bị; thông tin phương tiện được đồng bộ với Hệ thống Quản lý Gara của đối tác (Garage API) qua giao thức REST.

4.2.3 Tìm xe (Find My Car)

Khi người dùng không nhớ vị trí phương tiện, hệ thống hỗ trợ tìm kiếm theo hai cấp độ tương ứng với khoảng cách:

- **Định vị từ xa qua bản đồ UWB:** vị trí xe được hiển thị liên tục trên bản đồ isometric 3D (Mục 4.2.1), cho phép xác định tầng và vùng đỗ xe ngay từ màn hình chính. Chức năng này đã triển khai đầy đủ trong phạm vi đề tài.
- **Dẫn hướng cận cảnh — giải pháp đề xuất (UWB + BLE):** để dẫn người dùng tiếp cận chính xác đến ô đỗ ở cự ly gần, đề tài đề xuất trang bị thêm một Tag UWB di động (người tìm xe cầm hoặc đeo). Tag này hoạt động như một nút khảo sát (survey tag) trong hạ tầng Anchor sẵn có: IoT Server trilaterate đồng thời vị trí xe (Tag trên xe) và vị trí người tìm (Tag cầm tay), sau đó tính vector khoảng cách và phương hướng từ người đến xe, hiển thị mũi tên chỉ đường trực tiếp trên bản đồ. Cơ chế này tận dụng lại toàn bộ hạ tầng Anchor và pipeline định vị đã xây dựng mà không cần lắp đặt thêm thiết bị nào khác. Hỗ trợ cho Tag UWB cầm tay, điện thoại có thể kết hợp quét tín hiệu BLE (Bluetooth Low Energy) từ Tag trên xe để ước lượng khoảng cách ở cự ly dưới 5m (nguyên lý RSSI beacon), tạo thành lớp dẫn hướng "bước cuối" (last-meter guidance) khi người dùng đã vào gần khu vực đỗ xe. Giải pháp kết hợp UWB + BLE này là hướng phát triển tiếp theo sau khi hoàn thiện phần cứng Tag cầm tay. Ý tưởng dẫn hướng cận cảnh bằng Tag UWB di động được minh họa trong Hình 4.2.

[Khu vực hình minh họa — ảnh chụp thực tế sẽ được bổ sung]

Hình 4.2: Giải pháp tìm xe bằng Tag UWB cầm tay

4.2.4 Khảo sát mặt bằng (Floor Survey — Walk & Press)

Tính năng này cho phép quản trị viên tạo bản đồ tầng hầm mới bằng cách trực tiếp đi khảo sát thực địa, thay vì phải đo vẽ thủ công:

- **Bước 1:** Chọn thiết bị Tag khảo sát
- **Bước 2:** Nhập thông tin tầng (tên, số tầng)
- **Bước 3:** Đứng sát từng Anchor bấm "Đánh dấu điểm" → server tự phát hiện 4 Anchor, suy luận tọa độ tuyệt đối bằng giao điểm hai đường tròn, trilaterate các điểm còn lại → dựng polygon khép kín
- **Bước 4:** Xem trước polygon và gán Anchor; hỗ trợ "Vòng 2+" để tinh chỉnh (chèn điểm vào cạnh gần nhất)
- **Bước 5:** Định nghĩa các ô đỗ xe (slot) trên bản đồ

4.2.5 Phát hiện đỗ sai quy định (Mispark Detection)

Module phát hiện đỗ sai quy định tự động quét trạng thái mỗi 20 giây. Một phương tiện đứng yên liên tục từ 3 phút trở lên (độ dịch chuyển dưới 1m) và nằm ngoài bán kính 2,2m của mọi ô đỗ hợp lệ sẽ được xác định là trường hợp "vi phạm nghi ngờ". Quản trị viên nhận thông báo và ra quyết định qua giao diện web theo một trong hai hướng: phê duyệt (hệ thống tạo ô đỗ mới có định danh AUTO_xxx tại vị trí đó) hoặc từ chối (hệ thống ghi nhận vào nhật ký vi phạm). Cơ chế này hạn chế cảnh báo sai bằng cách đưa con người vào vòng xác nhận thay vì tự động kết luận.

4.2.6 Cảnh báo an ninh: chống trộm và vùng giới hạn (geofence)

Hệ thống tích hợp hai cơ chế giám sát an ninh chủ động, chạy tự động song song với luồng định vị và được kích hoạt riêng lẻ theo từng tầng qua giao diện quản trị.

Chống trộm (Anti-theft detection): Server quét stream vị trí mỗi 10 giây và duy trì một máy trạng thái thuần (bộ xử lý trạng thái di chuyển) cho từng phương tiện. Một xe được xem là "đã đỗ ổn định" sau khi đứng yên liên tục từ 2 phút trở lên (độ dịch chuyển dưới 1m). Nếu xe đang ở trạng thái "đã đỗ" mà bỗng dịch chuyển hơn 1m trong một chu kỳ kiểm tra, hệ thống phát cảnh báo chủ xe ngay lập tức. Nhờ máy

trạng thái tách biệt hai giai đoạn (xe đang vào/di chuyển hợp lệ và xe đã đỗ), phương tiện đang tự di chuyển vào ô đỗ không bị báo nhầm. Cảnh báo có snooze 5 phút để tránh spam khi tín hiệu UWB dao động.

Vùng geofence: Quản trị viên định nghĩa các vùng đa giác trên bản đồ tầng theo ba loại: khu vực hạn chế (cấm tất cả xe), khu vực xe điện (EV) (chỉ xe điện), và VIP (chỉ xe ưu tiên). Mỗi chu kỳ kiểm tra, server áp dụng thuật toán điểm-trong-đa-giác (phép kiểm tra điểm thuộc đa giác) và hàm phép phát hiện vi phạm vùng để kiểm tra tất cả xe đang hiện diện trên tầng: xe đi vào vùng khu vực hạn chế, hoặc xe không có quyền đi vào vùng khu vực xe điện (EV)/VIP → phát cảnh báo vi phạm. Cảnh báo được chuyển qua dịch vụ thông báo (Redis + MQTT) và lưu vào nhật ký theo tầng với snooze 5 phút. Giao diện web bảng điều khiển geofence cho phép bật/tắt tính năng, xem feed cảnh báo theo thời gian thực và quản lý danh sách vùng. Toàn bộ logic địa lý được tách thành hàm thuần (mô-đun geofence) với 9 kiểm thử đơn vị độc lập, đảm bảo tính đúng đắn của phép tính không phụ thuộc vào phần cứng.

4.2.7 Chế độ Gara (Standard / Valet)

Hệ thống hỗ trợ hai chế độ vận hành nhằm thích ứng với các mô hình tổ chức bãi đỗ khác nhau:

- **Chế độ tiêu chuẩn (Standard):** mỗi Tag được gắn cố định với một phương tiện cụ thể và theo phương tiện đó trong suốt thời gian đỗ; số lượng Tag tương ứng với số phương tiện.
- **Chế độ đỗ xe hộ (Valet):** mỗi Tag được gắn với một nhân viên thay vì phương tiện; tại thời điểm nhân viên giao trả chìa khóa, hệ thống ghi nhận (snapshot) tọa độ phương tiện và liên kết tọa độ đó với phương tiện, sau đó giải phóng Tag để nhân viên tiếp tục phục vụ phương tiện kế tiếp. Nhờ vậy, số lượng Tag cần thiết nhỏ hơn nhiều so với số phương tiện được phục vụ.

4.2.8 Đề xuất bố trí tự động từ dữ liệu vận hành (ô đỗ và lối đi)

Hệ thống tích hợp hai tính năng phân tích dữ liệu định vị tích lũy để tự suy luận bố trí bãi đỗ, giảm công tác thiết kế thủ công:

Đề xuất ô đồ tự động (cơ chế tự động phát hiện ô đồ): Từ bảng lịch sử vị trí dữ liệu lịch sử vị trí phương tiện, hệ thống phát hiện các sự kiện "dwell" (xe đứng yên đủ lâu), gom cụm bằng thuật toán mean-shift để tìm tâm tập trung, rồi suy ra tâm và hướng của ô đồ bằng phân tích thành phần chính (PCA). Kết quả là tập ứng viên ô đồ có định danh tự động ô đồ tự sinh, được trình bày ở chế độ xem trước (dryRun) trước khi quản trị viên duyệt và lưu vào cơ sở dữ liệu. Cơ chế duyệt thủ công đảm bảo con người kiểm soát chất lượng trước khi ô đồ trở thành dữ liệu chính thức.

Đề xuất lối đi (mô-đun suy luận lối đi): Tách điểm xe đang di chuyển (tốc độ vượt ngưỡng tối thiểu, loại bỏ điểm đỗ tĩnh) → rasterize lưới mật độ (ô 0,5m) → gom các ô liền kề theo 8-connectivity thành vùng hành lang kèm heatmap. Thuật toán lọc theo biên đa giác tăng để chỉ giữ lại vùng bên trong bãi đỗ. Endpoint API suy luận lối đi trả về danh sách vùng lối đi; giao diện web vẽ vùng lối đi màu tím chồng lên bản đồ ô đồ kèm chú thích. Toàn bộ logic được tách thành 6 kiểm thử đơn vị vitest. Kết hợp hai tính năng này, bản đồ bãi đỗ có khả năng tự học dần từ dữ liệu vận hành thực tế mà không cần đo vẽ lại thủ công.

4.2.9 Thông báo đẩy và thông báo trong ứng dụng

Hệ thống phát thông báo theo thời gian thực qua hai kênh tương ứng với mức độ ưu tiên của sự kiện: thông báo đẩy (push notification) qua Firebase Cloud Messaging (FCM) dành cho các sự kiện quan trọng cần người dùng chú ý ngay cả khi không mở ứng dụng, chẳng hạn xe vào/ra hoặc cảnh báo đỗ sai quy định; và thông báo trong ứng dụng dạng banner dành cho các cập nhật trạng thái thứ yếu. Toàn bộ thông báo được lưu trữ để người dùng tra cứu lại theo lịch sử.

4.2.10 Giao diện người dùng

Ứng dụng hỗ trợ giao diện chế độ tối (Dark Mode) và chế độ sáng (Light Mode), tự động chuyển đổi theo thiết lập của hệ điều hành. Giao diện được tổ chức theo một hệ thống thiết kế thống nhất dựa trên bộ thẻ định danh thuộc tính giao diện (ThemePalette tokens), qua đó duy trì tính nhất quán về màu sắc và bố cục trên toàn bộ các màn hình. Các màn hình chính của ứng dụng được tóm tắt trong bảng dưới đây:

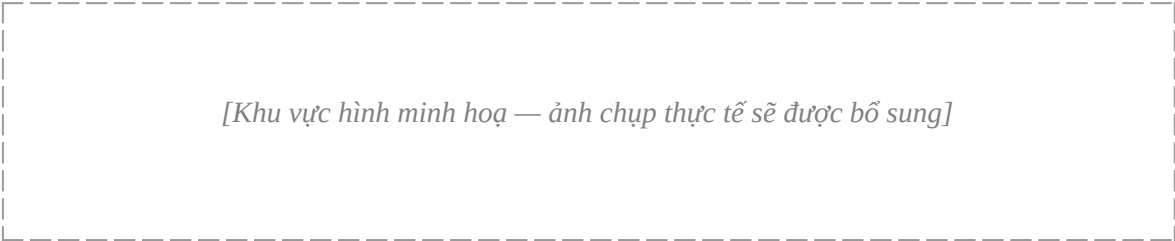
Màn hình	Mô tả
Home	Dashboard tổng quan, bản đồ tầng hầm 3D
Vehicles	Danh sách xe, thêm xe, QR scan
IoT Floor Map	Bản đồ UWB real-time, panel chi tiết xe
Chatbot	Hội thoại AI, lịch sử phiên
Notifications	Lịch sử thông báo
Settings	Cấu hình ứng dụng, theme
Floor Survey	Khảo sát mặt bằng walk-and-press
Positioning Boards	Quản lý bảng định vị UWB

4.3 Tính năng Chatbot AI

Chatbot AI là lớp giao tiếp ngôn ngữ tự nhiên giúp người dùng truy vấn dữ liệu của hệ thống mà không cần thao tác qua nhiều màn hình. Mục này trình bày khả năng hội thoại bằng tiếng Việt, cơ chế suy luận–gọi công cụ theo vòng lặp agentic, cách trình bày kết quả bằng thành phần giao diện chuyên biệt, cơ chế lưu trữ phiên hội thoại và giải pháp dự phòng nhà cung cấp mô hình ngôn ngữ lớn.

4.3.1 Hội thoại tự nhiên bằng tiếng Việt

Chatbot cho phép truy vấn thông tin gara bằng ngôn ngữ tự nhiên tiếng Việt, chẳng hạn các câu hỏi "Xe biển số 51A-123.45 đang ở đâu?", "Còn bao nhiêu chỗ trống tầng B1?" hoặc "Lịch sử bảo dưỡng xe của tôi tháng trước". Hệ thống phân tích ý định của người dùng (intent) và ánh xạ tới công cụ truy vấn dữ liệu phù hợp, qua đó trả về thông tin chính xác từ nguồn dữ liệu thực thay vì sinh nội dung suy đoán — đúng theo nguyên lý tăng cường truy xuất (Retrieval-Augmented Generation – RAG) nhằm hạn chế hiện tượng "ảo giác" (hallucination) của mô hình ngôn ngữ lớn [3]. Giao diện hội thoại của chatbot trên ứng dụng di động được minh họa trong Hình 4.3.



Hình 4.3: Màn hình chatbot AI tiếng Việt

4.3.2 Agentic Loop và MCP Tools

Dịch vụ chatbot thực hiện vòng lặp tác tử (agentic) tối đa 5 lần lặp: nhận câu hỏi → gửi cho LLM kèm danh sách công cụ → LLM trả về yêu cầu gọi công cụ (dạng JSON có cấu trúc) → MCP Server thực thi công cụ → trả kết quả về LLM → LLM tổng hợp câu trả lời cuối. Các công cụ truy vấn hiện có:

Công cụ (chức năng)	Dữ liệu trả về
Trạng thái bãi đỗ	Mức lấp đầy và số ô trống theo từng tầng
Định vị xe	Vị trí hiện tại và trạng thái xe theo biển số
Hồ sơ bảo dưỡng	Lịch sử bảo dưỡng của xe
Tổng quan gara	Số liệu tổng hợp toàn gara
Xe đang đỗ	Danh sách phương tiện đang đỗ
Danh mục tầng	Danh sách các tầng hầm
Lịch sử di chuyển	Hành trình di chuyển của xe
Tra cứu tri thức	Tìm kiếm ngữ nghĩa trong kho tri thức nội bộ

4.3.3 Thẻ giao diện trực quan

Bên cạnh câu trả lời dạng văn bản, mỗi loại dữ liệu trả về được trực quan hóa bằng một thẻ giao diện chuyên biệt nhằm tăng khả năng đọc hiểu và tra cứu nhanh:

- **Thẻ tổng quan tầng:** tổng quan tầng (tổng số ô, đang dùng, còn trống, tỷ lệ lấp đầy)

- **Thẻ vị trí xe:** vị trí xe trên bản đồ thu nhỏ kèm tọa độ X/Y
- **Lưới trạng thái ô đồ:** lưới ô đồ tô màu theo trạng thái
- **Thẻ thông tin xe:** thông tin chi tiết xe kèm lịch sử

4.3.4 Lịch sử hội thoại đa phiên

Mỗi phiên hội thoại được lưu trữ trên Redis với thời gian sống (Time-To-Live – TTL) 7 ngày và được đồng bộ về ứng dụng di động. Người dùng có thể tiếp tục bất kỳ phiên nào từ màn hình lịch sử hoặc khởi tạo phiên mới; hệ thống hiển thị tối đa 20 phiên gần nhất nhằm cân bằng giữa khả năng truy hồi ngữ cảnh và hiệu năng tải dữ liệu.

4.3.5 Dual-Provider LLM với Fallback

Mô hình ngôn ngữ lớn (Large Language Model – LLM) chính được sử dụng là Gemini 2.5 Flash Lite, với hạn mức 500 yêu cầu/ngày ở gói miễn phí. Khi gặp lỗi vượt hạn mức (mã 429) hoặc lỗi phía máy chủ (nhóm mã 5xx), hệ thống tự động chuyển sang nhà cung cấp dự phòng DeepSeek nhằm duy trì tính sẵn sàng của dịch vụ mà người dùng không cảm nhận được sự gián đoạn. Cơ chế dự phòng (fallback) được điều khiển hoàn toàn qua biến môi trường, cho phép thay đổi danh sách nhà cung cấp mà không phải can thiệp vào mã nguồn.

4.3.6 Đánh giá chất lượng truy hồi (Hybrid RAG)

Để kiểm chứng đóng góp IoT Snapshot Documents, khóa luận xây dựng một harness đánh giá truy hồi offline, chạy ngay trên hàm tìm kiếm lai (Hybrid Search) của hệ thống vận hành. Quá trình đo hoàn toàn cục bộ (embedding bằng mô hình embeddinggemma 768 chiều trên Ollama), với corpus 15 tài liệu tĩnh cùng các snapshot IoT tích lũy và 13 câu hỏi tiếng Việt được gán nhãn thủ công (bộ nhãn minh họa). Kết quả cho thấy truy hồi ngữ nghĩa là yếu tố quyết định: cấu hình Hybrid đạt **Recall@5 = 0,821**, **nDCG@5 = 0,809** và **MRR = 0,962**, trong khi chỉ dùng BM25 đạt 0,026 — chênh hơn 30 lần. Ở các câu hỏi về anchor và thiết bị offline, các snapshot cảnh báo IoT chen vào hạng 2–5 của kết quả, xác nhận snapshot IoT thực sự tham gia cạnh tranh xếp hạng cùng tài liệu tĩnh. Về độ trễ truy hồi (đo trên cùng

harness), nhánh BM25 thuần chỉ tốn khoảng 1,4 ms (trung vị p50), nhánh vector khoảng 211 ms, còn truy vấn lai khoảng 219 ms (p50) và 297 ms (bách phân vị 95 – p95); khi bật lọc độ-cũ (TTL), p95 tăng lên khoảng 1185 ms do thêm điều kiện thời gian vào cả hai nhánh tìm kiếm. Độ trễ bị chi phối chủ yếu bởi bước sinh embedding cho câu hỏi trên CPU (khoảng 100 ms mỗi lần) và kích thước corpus trong phép hợp nhất; dùng dịch vụ embedding đám mây dự kiến giảm khoảng một nửa.

4.3.7 Đánh giá chất lượng trả lời theo môi trường (none / MCP / MCP+RAG)

Để định lượng đóng góp của từng tầng hỗ trợ, khóa luận so sánh ba môi trường vận hành trên cùng bộ **n=101** câu hỏi (40 câu tri thức tĩnh G1 + 61 câu trạng thái-IoT thay đổi nhanh G2, đáp án vàng chốt từ cơ sở dữ liệu lúc hỏi), dùng mô hình sinh DeepSeek-V3.2 chat (temperature 0). Điểm mẫu chốt là cách chấm: thí nghiệm chấm **hoàn toàn tự động và khách quan** bằng đối chiếu dữ kiện cốt lõi của đáp án chuẩn (key-fact recall) — không dùng LLM làm giám khảo nên loại bỏ hoàn toàn self-preference bias và tái lập được 100%. Ở môi trường có công cụ, mô hình tự quyết định gọi MCP tool (agentic tool-calling) trên cơ sở dữ liệu kịch bản. Kết quả ở Bảng 4.1.

Bảng 4.1: So sánh độ chính xác và chi phí token của chatbot theo môi trường
(DeepSeek-V3.2 chat, chấm tự động khách quan, n=101)

Môi trường	G1 (n=40)	G2 (n=61)	Tổng (n=101) [95% CI]	Component-acc	Token TB/câu
Không dùng gì (none)	20,0%	1,6%	8,9% [5–16]	—	272
Có MCP (tool)	10,0%	93,4%	60,4% [51–69]	100,0%	1585
MCP + RAG (tool+RAG)	100,0%	100,0%	100,0% [96–100]	37,9%	1871

Kết quả bộc lộ một chuỗi hiệu ứng rõ ràng. Môi trường **none** gần như vô dụng với nhóm trạng thái-IoT (G2 chỉ đạt 1,6%) vì thông tin thời gian chạy không nằm trong tham số mô hình. Việc trang bị **MCP** cứu vớt G2 lên 93,4% nhờ khả năng truy vấn dữ liệu thật, nhưng tạo ra một *nghịch lý định tuyến công cụ*: G1 lại tụt còn 10,0% do mô hình cố gọi công cụ ngay cả với câu tri thức tĩnh. **MCP+RAG bù đắp hoàn toàn** cả hai chiều, đạt 100,0% [Wilson 95% CI: 96–100%] — khoảng tin cậy này không giao với none (5–16%) nên kết luận có ý nghĩa thống kê. Hai cơ chế MCP và RAG bổ sung chứ không thay thế nhau: MCP cần cho truy vấn trạng thái-IoT có cấu trúc, còn RAG cần cho tri thức tĩnh và hạn chế nghịch lý định tuyến. Về độ tin cậy của định tuyến tìm kiếm, mô hình không bịa tên hàm ngoài danh mục công cụ (tỷ lệ bịa công cụ 0,0%) và luôn chọn đúng công cụ khi quyết định gọi (độ chính xác chọn công cụ 100,0% ở môi trường MCP). Đáng chú ý, tỷ lệ phát lời gọi công cụ giảm từ 59,4% ở môi trường MCP xuống còn 21,8% ở môi trường MCP+RAG: do ngữ cảnh Hybrid RAG đã chứa sẵn IoT Snapshot nên mô hình thường trả lời trực tiếp từ ngữ cảnh thay vì phải gọi công cụ, trong khi độ chính xác tổng thể vẫn đạt 100% — minh chứng cho luận điểm IoT Snapshot Documents giúp hệ thống bớt phụ thuộc vào việc định tuyến công cụ của mô hình ngôn ngữ cho câu hỏi trạng thái. Chi phí token tăng dần 272 → 1585 → 1871 theo mức hỗ trợ.

Hai thí nghiệm phụ củng cố các kết luận trên. Thứ nhất, phép **cắt bỏ TTL** (ablation) định lượng trực tiếp giá trị của IoT Snapshot Documents: trên nhóm G2, dùng snapshot tươi đạt 82,0% trong khi snapshot cũ (vô hiệu TTL) chỉ đạt 75,4% — chênh 6,6 điểm phần trăm, cho thấy độ tươi của dữ liệu IoT là yếu tố có đóng góp đo được. Thứ hai, so sánh với biến thể **DeepSeek-V3.2 reasoner** cho thấy mô hình suy luận không vượt trội trong tác vụ tra cứu – gọi công cụ: ở môi trường MCP+RAG reasoner đạt 98,0% (kém chat 2 điểm) nhưng tốn nhiều token hơn (+29,5%). Do đó, với chatbot gara, ưu tiên mô hình non-thinking để tiết kiệm chi phí mà không đánh đổi chất lượng. *Các kết quả dựa trên bộ câu hỏi tổng hợp n=101 nhằm minh họa quy trình đánh giá; kết luận tổng quát cần bộ câu hỏi từ người dùng thực.*

4.4 Tính năng IoT Admin (Web Dashboard)

Giao diện web quản trị hướng tới đối tượng quản trị viên hệ thống, cung cấp các công cụ giám sát tập trung, hiệu chỉnh hạ tầng định vị và kiểm thử mà không cần phụ thuộc hoàn toàn vào phần cứng thực. Các chức năng chính bao gồm:

- **Live Tracker:** bản đồ isometric 3D hiển thị vị trí toàn bộ phương tiện đang đỗ, cập nhật theo thời gian thực qua SSE; hỗ trợ thao tác phóng to/thu nhỏ, di chuyển khung nhìn và kéo–thả Tag/Anchor để hiệu chỉnh tọa độ.
- **MQTT Simulator Sandbox:** môi trường mô phỏng dữ liệu UWB phục vụ kiểm thử khi chưa có phần cứng thực; hỗ trợ kịch bản nhiều phương tiện di chuyển theo quỹ đạo hình chữ L trong khoảng thời gian 24 giờ ảo được nén tỷ lệ thành 180 giây thực, qua đó kiểm chứng hành vi hệ thống trong điều kiện tải gần thực tế.
- **Floors Management:** quản lý thông tin tầng hầm và bật/tắt chức năng phát hiện đỗ sai quy định (Mispark Detection) theo từng tầng.
- **Mispark Panel:** theo dõi hàng chờ các trường hợp vi phạm nghi ngờ, phê duyệt hoặc từ chối từng trường hợp và tra cứu lịch sử vi phạm.
- **Devices:** danh sách thiết bị IoT kèm trạng thái trực tuyến/ngoại tuyến và lịch sử tín hiệu heartbeat (ping).
- **Báo cáo & phân tích vận hành:** thống kê thời gian lưu bãi (dwell-time), bản đồ nhiệt mức độ sử dụng (occupancy heatmap) và lưu lượng đỉnh (peak occupancy) tổng hợp từ lịch sử vị trí (trình bày ở Mục 4.7).

4.5 Pipeline xử lý dữ liệu định vị thời gian thực

Trong toàn bộ hệ thống, đường ống xử lý dữ liệu định vị thời gian thực là thành phần IoT cốt lõi: nó biến chuỗi bản tin khoảng cách thô do Tag UWB phát ra thành tọa độ phương tiện mượt và ổn định hiển thị trên bản đồ. Mục này trình bày cách hiện thực hóa đường ống đó trên IoT Server, từ khâu thu thập qua MQTT, định vị bằng trilateration, làm trơn và tiết lưu cập nhật cho đến phân phối kết quả; các tham số thiết kế nêu trong mục được đối chiếu trực tiếp với hệ thống đang vận hành (mô-đun xử lý luồng vị trí) chứ không phải số liệu giả định.

4.5.1 Tổng quan đường ống xử lý

Mỗi giây, Tag gắn trên xe đo khoảng cách tới bốn Anchor cố định và công bố một bản tin duy nhất lên chủ đề MQTT bản tin định vị gửi qua MQTT. IoT Server đăng ký nhận toàn bộ luồng này và xử lý theo từng bản tin một cách không trạng thái (stateless): với mỗi bản tin, hệ thống lần lượt (i) tra cứu tọa độ Anchor của tầng từ bộ nhớ đệm trong RAM, được làm mới mỗi 15 giây vì tọa độ Anchor hầu như không đổi; (ii) giải bài toán trilateration bình phương tối thiểu để suy ra tọa độ (x, y) kèm sai số dư; (iii) loại bỏ lời giải có sai số vượt ngưỡng tin cậy; (iv) đưa tọa độ qua bộ lọc One-Euro để khử dao động; (v) chỉ phát đi cập nhật khi vị trí thay đổi đáng kể nhằm tiết kiệm băng thông; và (vi) ghi kết quả vào Redis rồi đẩy đồng thời tới ứng dụng di động qua Server-Sent Events (SSE) và tới hệ quản lý gara của đối tác qua Socket.IO. Toàn bộ luồng dữ liệu này đã được mô tả trực quan ở Hình 3.5. Các tham số thiết kế then chốt của từng khâu được tổng hợp trong Bảng 4.2.

Bảng 4.2: Tham số thiết kế của pipeline định vị thời gian thực (đối chiếu trực tiếp thông số thiết kế của mô-đun xử lý luồng vị trí)

Khâu xử lý	Tham số thiết kế	Giá trị
Thu thập (MQTT)	Tần suất Tag công bố bản tin /uwb	1 Hz (mỗi giây)
Thu thập (MQTT)	Chu kỳ heartbeat /ping (xác định hiện diện)	30 s (TTL 35 s)
Định vị	Thuật toán	Trilateration bình phương tối thiểu, không trạng thái
Định vị	Ngưỡng loại bỏ theo sai số dư (RMS residual)	0,8 m (80 cm)
Làm trơn	Bộ lọc One-Euro — minCutoff	0,5
Làm trơn	Bộ lọc One-Euro — β	0,1
Làm trơn	Ngưỡng "ghim" khi đứng yên (snap)	0,03 m (3 cm)

Tiết lưu cập nhật	Phát cập nhật khi dịch chuyển vượt	0,10 m (10 cm)
Tiết lưu cập nhật	...hoặc khi quá thời gian (heartbeat)	30 s
Bộ nhớ đệm	TTL phát hiện mất tín hiệu (stale)	5 s
Bộ nhớ đệm	TTL vị trí gần nhất (khôi phục nhanh)	24 giờ
Bộ nhớ đệm	TTL cache tọa độ Anchor trong RAM	15 s
Phân phối	Kênh tới ứng dụng di động	Server-Sent Events (SSE)
Phân phối	Kênh tới hệ quản lý đối tác (GMS)	Socket.IO
Phân phối	Mục tiêu thiết kế độ trễ đầu-cuối	< 200 ms

4.5.2 Trilateration không trạng thái và loại bỏ lời giải nhiễu

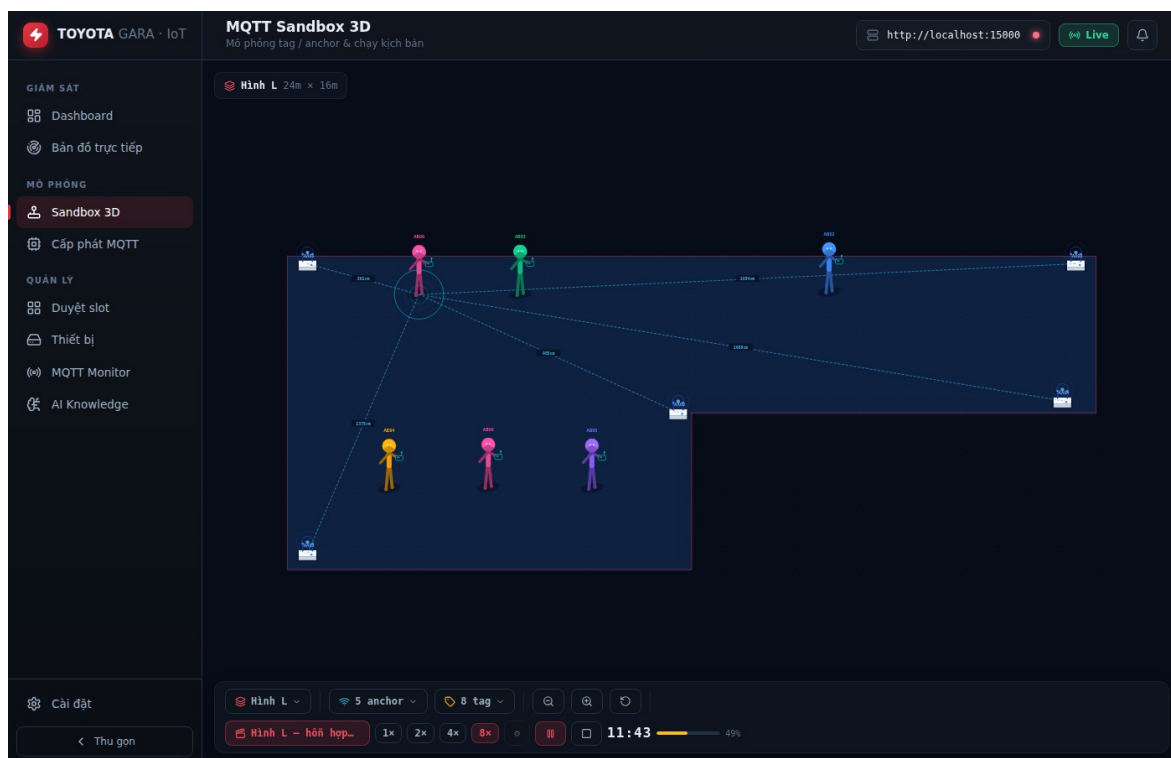
Khác với cách tiếp cận tích lũy khoảng cách từng Anchor theo thời gian, hệ thống chọn xử lý không trạng thái: mỗi bản tin bản tin định vị đã mang đầy đủ khoảng cách tới cả bốn Anchor nên có thể định vị độc lập ngay lập tức mà không phụ thuộc các bản tin trước. Cách làm này giúp đường ống đơn giản, không tích lũy sai số theo thời gian và tự phục hồi sau gián đoạn. Lời giải trilateration đi kèm một ước lượng độ tin cậy là căn bậc hai trung bình bình phương của phần dư (RMS residual, đơn vị xen-ti-mét); nếu sai số này vượt 0,8m, hệ thống coi lời giải là không đáng tin — thường do hình học điểm đo xấu hoặc nhiễu NLOS mạnh — và bỏ qua bản tin đó thay vì hiển thị một vị trí sai lệch. Cơ chế lọc theo độ tin cậy này giữ cho phương tiện không "nhảy" loạn xạ trên bản đồ trong vùng tín hiệu kém, đánh đổi một phần tần suất cập nhật để lấy độ ổn định hiển thị.

4.5.3 Làm trơn, tiết lưu cập nhật và phát hiện mất tín hiệu

Tọa độ thô sau trilateration vẫn còn dao động nhẹ ngay cả khi xe đứng yên, do đó được đưa qua bộ lọc One-Euro [2] với tham số tần số cắt tối thiểu 0,5 Hz và hệ số $\beta = 0,1$ — cấu hình ưu tiên khử nhiễu khi tĩnh nhưng vẫn bám sát khi xe di chuyển. Khi sai khác giữa hai mẫu liên tiếp nhỏ hơn 3cm, hệ thống "ghim" vị trí (snap) để loại bỏ trôi dạt tàn dư. Để tiết kiệm băng thông và giảm tải cho phía hiển thị, một cập nhật chỉ được phát đi khi phương tiện dịch chuyển quá 10cm so với lần phát trước, hoặc khi đã quá 30 giây kể từ cập nhật gần nhất (nhịp heartbeat). Song song, mỗi khi nhận bản tin, hệ thống làm mới một khóa Redis có thời gian sống 5 giây; nếu khóa này hết hạn, Tag được xem là mất tín hiệu (stale) và phương tiện được hiển thị mờ trên bản đồ thay vì biến mất đột ngột, giúp người dùng phân biệt giữa "xe đã rời đi" và "xe tạm mất sóng". Vị trí gần nhất được lưu trong 24 giờ để khôi phục nhanh khi tải lại, còn bộ lọc One-Euro được khởi tạo lại nếu khoảng trống tín hiệu vượt 15 giây nhằm tránh kéo lê trạng thái cũ.

4.5.4 Kiểm chứng vận hành trên môi trường mô phỏng (Sandbox)

Do việc kiểm thử trên phần cứng đầy đủ tốn kém và khó lặp lại, đường ống xử lý được kiểm chứng vận hành trên môi trường mô phỏng MQTT Simulator Sandbox của giao diện quản trị (Mục 4.4). Sandbox phát sinh dữ liệu UWB theo một kịch bản tải gần thực tế: mặt bằng hình chữ L với năm Anchor và tám Tag phương tiện di chuyển theo quỹ đạo trong 24 giờ ảo được nén tỷ lệ thành 180 giây thực. Quá trình kiểm chứng được thực hiện tự động bằng trình duyệt không giao diện (Playwright headless): sau khi nạp và phát kịch bản, hệ thống dựng đúng sàn hình L, hiển thị đủ năm Anchor và tám Tag, đồng hồ ảo chạy đúng nhịp, các phương tiện lần lượt đến và đỗ vào ô theo lịch trình mà không phát sinh lỗi JavaScript nào trên trình duyệt. Kết quả này xác nhận đường ống từ khâu nhận bản tin đến khâu hiển thị hoạt động ổn định dưới điều kiện nhiều phương tiện đồng thời, làm tiền đề cho việc triển khai trên phần cứng thực (Hình 4.4).



Hình 4.4: Mô phỏng Sandbox — kịch bản nhiều xe (mặt bằng hình L)

4.6 Phương pháp đánh giá độ chính xác định vị UWB

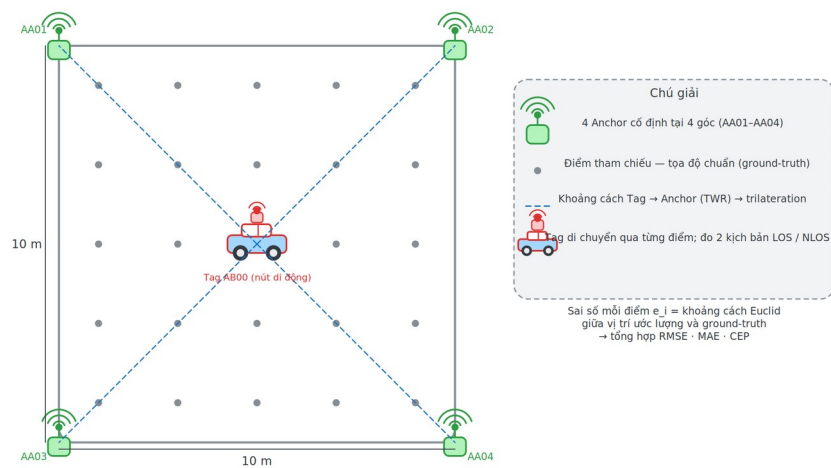
Nhằm kiểm chứng chất lượng định vị của hệ thống một cách khách quan, nghiên cứu này thiết kế một quy trình đánh giá định lượng dựa trên các điểm tham chiếu có tọa độ thực đã biết (ground-truth). Mục này trình bày **bố trí thực nghiệm** và **phương pháp đo dự kiến**; các số liệu định lượng thu được sẽ được báo cáo sau khi hoàn tất quá trình đo đạc trên hệ thống thực tế.

4.6.1 Bố trí thực nghiệm

Kế hoạch đo được thiết lập trong một không gian bãi đỗ xe có kích thước $10\text{ m} \times 10\text{ m}$, với bốn Anchor cố định (AA01–AA04) đặt tại bốn góc nhằm bao phủ toàn bộ vùng định vị và bảo đảm cấu hình hình học cân đối cho phép trilateration. Tag (AB00) đóng vai trò nút di động được đặt lần lượt tại một tập hợp điểm tham chiếu phân bố đều trên mặt sàn; tọa độ thực của mỗi điểm được đo bằng thước/lưới định vị trên mặt bằng và dùng làm giá trị chuẩn để đối chiếu. Bố trí này được trình bày trực quan trong Hình 4.5.

Để phản ánh điều kiện vận hành thực tế của bãi đỗ xe, quá trình đo được lặp lại trong hai kịch bản truyền sóng: (i) tầm nhìn thẳng (Line-of-Sight – LOS), khi không có vật cản giữa Tag và các Anchor; và (ii) tầm nhìn bị che khuất (Non-Line-of-Sight – NLOS), khi xuất hiện vật cản (thân xe kim loại) làm suy giảm và phản xạ tín hiệu. Việc tách hai kịch bản cho phép định lượng riêng ảnh hưởng của hiện tượng đa đường (multipath) lên sai số định vị.

BỐ TRÍ THỰC NGHIỆM ĐÁNH GIÁ ĐỘ CHÍNH XÁC ĐỊNH VỊ UWB



Hình 4.5: Bố trí thực nghiệm đánh giá định vị UWB

4.6.2 Quy trình và chỉ tiêu đo

Tại mỗi điểm tham chiếu, hệ thống ghi nhận đồng thời hai chuỗi tọa độ ước lượng: vị trí **thô** (raw) thu trực tiếp từ bước trilateration và vị trí **đã làm trơn** (filtered) sau khi đi qua bộ lọc One-Euro [2]. Việc đo song song hai chuỗi này cho phép đánh giá riêng đóng góp của khâu hậu xử lý đối với việc giảm dao động (jitter) khi Tag đứng yên, đồng thời kiểm tra xem bộ lọc có gây trễ hoặc lệch hệ thống (bias) đáng kể hay không.

Các chỉ tiêu đánh giá dự kiến gồm:

- **Sai số vị trí:** khoảng cách Euclid giữa tọa độ ước lượng và tọa độ ground-truth tại từng điểm, tổng hợp trên toàn bộ tập điểm cho cả hai chuỗi (thô và đã lọc), trong hai điều kiện LOS và NLOS.

- **Mức độ dao động khi tĩnh:** độ phân tán của chuỗi tọa độ ước lượng khi Tag giữ nguyên vị trí, dùng để định lượng hiệu quả khử nhiễu của bộ lọc One-Euro.
- **Độ trễ đầu–cuối (end-to-end latency):** thời gian từ thời điểm Tag công bố bản tin khoảng cách qua MQTT [8] đến thời điểm vị trí tương ứng được đẩy tới phía hiển thị qua Server-Sent Events (MQTT → trilateration → One-Euro → SSE).

Để bảo đảm tính tái lập, quy trình đánh giá được hiện thực hóa thành một công cụ chạy bằng dòng lệnh trên IoT Server: công cụ nạp dữ liệu đo, đưa qua **đúng** chuỗi xử lý của hệ thống đang vận hành (cùng hàm trilateration và bộ lọc One-Euro với cùng tham số tần số cắt tối thiểu 0,5 Hz, hệ số $\beta = 0,1$), sau đó tự động tính RMSE, MAE và CEP theo từng điều kiện truyền sóng (LOS/NLOS) và từng khâu xử lý (thô/đã lọc), rồi xuất ra bảng kết quả. Nhờ đó, khi có dữ liệu đo thực, các số liệu định lượng được tạo ra một cách nhất quán và có thể kiểm chứng lại.

4.6.3 Định nghĩa hình thức các độ đo sai số định vị

Để bảo đảm tính khách quan và khả năng so sánh với các nghiên cứu cùng lĩnh vực, sai số định vị được lượng hóa bằng ba độ đo chuẩn mực trong tài liệu định vị trong nhà [6]: căn bậc hai sai số trung bình bình phương (RMSE), sai số tuyệt đối trung bình (MAE) và bán kính sai số xác suất tròn (CEP). Gọi N là số điểm tham chiếu; (x_i, y_i) là tọa độ ước lượng và (x_i, y_i) là tọa độ chuẩn (ground-truth) tại điểm thứ i ; đặt $e_i = \sqrt{(x_i - x_i)^2 + (y_i - y_i)^2}$ là sai số khoảng cách Euclid (lỗi định vị 2D) tại điểm đó.

Căn bậc hai sai số trung bình bình phương (Root Mean Square Error – RMSE) đo độ lớn tổng thể của sai số; do lấy bình phương trước khi trung bình, RMSE nhạy với các sai số lớn (outlier) nên phản ánh tốt độ ổn định của hệ thống định vị:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N ((x_i - x_i)^2 + (y_i - y_i)^2)} = \sqrt{\frac{1}{N} \sum_{i=1}^N e_i^2}$$

Sai số tuyệt đối trung bình (Mean Absolute Error – MAE) là trung bình cộng của sai số khoảng cách, biểu thị mức sai lệch điển hình theo đơn vị xen-ti-mét và ít chịu ảnh hưởng của outlier hơn RMSE:

$$MAE = \frac{1}{N} \sum_{i=1}^N e_i = \frac{1}{N} \sum_{i=1}^N \sqrt{(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2}$$

Bán kính sai số xác suất tròn (Circular Error Probable – CEP) là bán kính của đường tròn tâm tại điểm chuẩn chứa 50% số mẫu đo, tức trung vị (median) của tập sai số khoảng cách $\{e_i\}$ [6]. CEP cho biết với một nửa số lần đo, vị trí ước lượng nằm trong vòng bán kính này quanh vị trí thật — một cách diễn đạt trực quan độ chính xác theo xác suất thường dùng trong định vị và dẫn đường:

$$CEP = \text{median}\{e_1, e_2, \dots, e_N\}$$

Phân vị 95% sai số định vị (P95) là giá trị phân vị thứ 95 của tập sai số khoảng cách $\{e_i\}$: có 95% số mẫu đo sai số nhỏ hơn hoặc bằng giá trị này. P95 phản ánh "trường hợp gần xấu nhất" của hệ thống định vị, bổ sung thông tin về đuôi phân bố mà RMSE và CEP không thể hiện rõ — đặc biệt quan trọng trong môi trường NLOS có sai số lớn không thường xuyên (outlier):

$$P95 = \text{percentile}_{95}\{e_1, e_2, \dots, e_N\}$$

Chú thích: Định nghĩa $CEP = \text{median}(\$e_i\$)$ chính xác khi phân bố lỗi 2D xấp xỉ đối xứng. Trong các tài liệu định vị và dẫn đường, CEP thường được tính dưới giả định phân bố Rayleigh (sai số mỗi trục Gaussian độc lập, phương sai bằng nhau), khi đó $CEP \approx 1,177\sigma$. Khi phân bố thực tế lệch đáng kể (ví dụ môi trường NLOS nhiều nhiễu đa đường), P95 là chỉ số bổ sung cần thiết để phản ánh đầy đủ đuôi phân bố sai số.

Bộ bốn độ đo này bổ trợ lẫn nhau: MAE phản ánh sai số điển hình, RMSE làm nổi bật trường hợp xấu, CEP diễn giải độ chính xác dưới dạng xác suất, còn P95 đặc trưng cho giới hạn an toàn của hệ thống; nhờ đó cho phép đánh giá đa chiều chất lượng định vị của hệ thống trước và sau bộ lọc One-Euro [2], trong cả hai điều kiện LOS và NLOS.

4.6.4 Kết quả kiểm chứng quy trình trên dữ liệu mô phỏng

Trong khi chờ dữ liệu đo thực địa, để xác nhận rằng toàn bộ chuỗi xử lý (trilateration → One-Euro Filter → tính chỉ số) vận hành đúng, khóa luận chạy harness đánh giá công cụ đánh giá định vị tự động trên tập dữ liệu **mô phỏng (synthetic)** được tạo theo đặc tả kỹ thuật của thiết bị UWB EWT550 trong hai điều kiện truyền sóng. Kết quả được trình bày trong Bảng 4.3 dưới đây.

Nhấn mạnh: đây là dữ liệu mô phỏng để kiểm chứng quy trình — CHƯA phải kết quả đo thực địa. Khi có phần cứng, chạy công cụ đánh giá định vị tự động với file JSON đo thực sẽ tạo ra bảng tương tự với số liệu thật.

Bảng 4.3: Kết quả định vị trên dữ liệu mô phỏng (minh họa quy trình — CHƯA phải đo thực địa), chạy qua đúng đường ống trilateration + One-Euro (tần số cắt tối thiểu 0,5 Hz; $\beta = 0,1$)

Nhóm	n	RMSE (cm)	MAE (cm)	CEP (cm)	P95 (cm)	Max (cm)
LOS / filtered	45	4,20	3,75	3,75	6,71	7,50
LOS / raw	45	4,74	4,23	4,18	7,75	8,31
NLOS / filtered	30	19,51	17,45	17,63	30,65	34,22
NLOS / raw	30	20,41	18,33	18,33	31,69	36,22
TỔNG — raw	75	13,42	9,87	5,76	29,46	36,22
TỔNG — filtered	75	12,76	9,23	5,30	27,87	34,22

Bộ mô phỏng xác nhận pipeline vận hành đúng hành vi kỳ vọng theo hai quan sát chính. Thứ nhất, **filtered luôn nhỏ hơn raw ở mọi nhóm** — chứng minh bộ lọc One-Euro có tác dụng giảm nhiễu thực chất: RMSE LOS giảm từ 4,74 → 4,20 cm

(-11,4%); RMSE NLOS giảm từ 20,41 → 19,51 cm (-4,4%). Thứ hai, **LOS đạt ~4 cm (cấp xen-ti-mét)** trong khi NLOS tăng lên ~20 cm — đúng như các nghiên cứu cảnh báo về suy giảm tín hiệu NLOS trong môi trường kim loại [6], [7]. Kết quả này làm cơ sở so sánh khi có dữ liệu thực địa (harness đã sẵn sàng, tham số cố định: tần số cắt tối thiểu 0,5 Hz, hệ số $\beta = 0,1$).

4.6.5 Ngưỡng cảnh hóa độ chính xác kỳ vọng theo tài liệu

Để đặt kết quả đo của hệ thống vào bối cảnh học thuật, Bảng 4.4 tổng hợp dải độ chính xác định vị điển hình của các công nghệ định vị trong nhà phổ biến theo các khảo sát uy tín [6], [7]. Số liệu trong bảng được trích từ tài liệu (không phải kết quả của nghiên cứu này) và dùng làm mốc tham chiếu để đối chiếu sai số đo được ở các mục sau. Có thể thấy UWB — nhờ đo trực tiếp thời gian bay của xung băng rộng theo chuẩn IEEE 802.15.4z [1] — đạt độ chính xác ở cấp xen-ti-mét, cao hơn một đến hai bậc độ lớn so với các công nghệ dựa trên cường độ tín hiệu (RSSI), qua đó lý giải về mặt định lượng lựa chọn UWB cho bài toán phân biệt từng ô đỗ xe.

Bảng 4.4: Độ chính xác định vị điển hình của các công nghệ theo tài liệu

Công nghệ	Nguyên lý đo điển hình	Độ chính xác theo tài liệu	Hoạt động trong nhà
UWB	Thời gian bay (ToF / TWR)	$\approx 10 - 30$ cm [6], [7]	Có
BLE (Bluetooth Low Energy)	Cường độ tín hiệu (RSSI)	$\approx 1 - 3$ m [6]	Có
Wi-Fi RTT (IEEE 802.11mc/FTM)	Thời gian bay (Fine Timing Measurement)	$\approx 1 - 2$ m [6]	Có
Wi-Fi RSSI (fingerprinting)	Cường độ tín hiệu (RSSI)	$\approx 2 - 6$ m [6]	Có
GPS / GNSS	Thời gian truyền từ vệ tinh	$\approx 3 - 5$ m (ngoài trời); không khả dụng trong nhà [6]	Không

Ghi chú: các giá trị trong Bảng 4.4 là khoảng độ chính xác điển hình được báo cáo trong tài liệu khảo sát [6], [7], có thể thay đổi tùy điều kiện triển khai (số lượng và hình học điểm tham chiếu, mức độ che khuất NLOS, hiệu chuẩn). Bảng này bổ sung cho Bảng 2.1 ở Chương 2 (so sánh tổng quan GPS/Wi-Fi-BLE/UWB) bằng cách tách riêng từng công nghệ và gắn nguồn định lượng cụ thể.

Trên cơ sở các độ đo và bố trí nêu trên, kết quả định lượng sẽ được tổng hợp thành bảng và đồ thị so sánh giữa hai điều kiện truyền sóng cũng như giữa hai khâu xử lý (trước và sau bộ lọc).

4.6.6 Kiến trúc pipeline định vị thời gian thực

Để đặt bối cảnh cho quá trình đánh giá, mục này mô tả từng tầng của chuỗi xử lý từ firmware đến màn hình hiển thị. Tất cả tham số dưới đây là tham số **thiết kế thực tế** xác nhận từ mã nguồn hệ thống — không phải kết quả đo.

Tầng 1 — Firmware → MQTT (1 Hz). Module Tag (AB00) đóng vai Initiator trong giao thức Two-Way Ranging của chuẩn IEEE 802.15.4z [1]: cứ mỗi giây, Tag thu khoảng cách tới toàn bộ Anchor (AA01–AA04) và đóng gói vào một bản tin MQTT duy nhất với topic `iot/{chip_id}/uwb`, payload `{tag_addr, ts, anchors: {AA01: d1, AA02: d2, AA03: d3, AA04: d4}}` — đơn vị cm. Thiết kế một bản tin/giây thay vì bốn bản tin riêng lẻ giảm overhead broker và bảo đảm bốn khoảng cách luôn đồng bộ thời điểm khi đưa vào Trilateration.

Tầng 2 — Trilateration stateless. IoT Server (NestJS) nhận bản tin, tra bảng `floors.anchor_positions` để lấy tọa độ cố định (x, y) của bốn Anchor theo đơn vị mét (cột `jsonb`), rồi giải hệ phương trình bình phương tối thiểu (4 ràng buộc khoảng cách, 2 ẩn x, y) mà không tích lũy trạng thái giữa các bản tin — mỗi bản tin xử lý hoàn toàn độc lập. Thiết kế stateless cho phép scale ngang (nhiều instance) mà không cần chia sẻ bộ nhớ trạng thái giữa các node.

Tầng 3 — Bộ lọc One-Euro. Tọa độ thô (x_{raw}, y_{raw}) được đưa qua bộ lọc One-Euro [2] với tham số `minCutoff = 0,5 Hz`, $\beta = 0,1$ — lọc mạnh khi xe đứng yên, giảm dần khi xe di chuyển nhanh. Mỗi thiết bị Tag duy trì riêng một đối tượng bộ lọc trong bộ nhớ của `PositioningService`, đảm bảo trạng thái lọc không bị lẫn

giữa các xe. Theo đặc tính thiết kế, lọc thêm khoảng 1 chu kỳ (≈ 1 s) trễ nhóm khi tín hiệu thay đổi chậm, và gần như không trễ khi tốc độ thay đổi lớn.

Tầng 4 — Throttle và Redis. Sau bộ lọc, hệ thống áp quy tắc tiết kiệm băng thông: chỉ ghi vào Redis và phát SSE khi xe di chuyển hơn **10 cm** so với tọa độ ghi gần nhất, hoặc kể từ lần gửi cuối đã qua hơn **30 s** (nhịp tim — đảm bảo client luôn nhận ít nhất 2 cập nhật/phút dù xe đứng yên). Song song, khóa `iot:tag:last_seen:{addr}` trong Redis được đặt TTL = **5 s**: khi khóa hết hạn (không nhận được bản tin trong 5 s), API REST snapshot trả xe ở trạng thái *stale* — xe vẫn hiển thị ở vị trí cuối với độ mờ giảm (opacity 42%) và chấm màu hồ phách để người dùng nhận biết tín hiệu gián đoạn.

Tầng 5 — SSE → Client. Tọa độ lọc được đẩy tới ứng dụng di động và giao diện web quản trị (IoT Admin) qua Server-Sent Events — kết nối HTTP/1.1 giữ mở, server chủ động gửi từng sự kiện dạng văn bản `data: {...}\n\n`. SSE được chọn thay WebSocket vì luồng dữ liệu một chiều (server → client), hoạt động qua proxy HTTP tiêu chuẩn và không yêu cầu thư viện client bổ sung. Đồng thời, dữ liệu vị trí **đã xử lý** (sau trilateration + One-Euro) được chuyển tiếp tới hệ thống quản lý gara (GMS) của đối tác qua Socket.IO — GMS chỉ nhận và hiển thị, không tự tính toán định vị.

Kiểm chứng tính ổn định runtime. Để xác nhận toàn bộ pipeline vận hành ổn định trong điều kiện nhiều thiết bị đồng thời, hệ thống được chạy thử trên kịch bản MQTT Simulator Sandbox: mặt bằng hình chữ L với **5 anchor** và **8 tag** (xe) hoạt động đồng thời, đồng hồ mô phỏng chạy tốc độ $8\times$, giao diện web quản trị hiển thị vị trí xe cập nhật liên tục. Kết quả kiểm tra bằng Playwright headless xác nhận **0 lỗi JavaScript** trên console sau toàn bộ kịch bản và các xe đến/đỗ đúng giờ mô phỏng như thiết kế (Hình 4.5). Kết quả này xác nhận pipeline không có lỗi đồng thời (race condition) hay rò rỉ bộ nhớ khi tải cao.

4.7 Phân tích và khai thác dữ liệu vận hành

Khi hệ thống được triển khai trên phần cứng thực và mặt bằng thật, dữ liệu đo tại các điểm tham chiếu sẽ được đưa qua đúng đường ống xử lý (trilateration → bộ lọc One-Euro) bằng công cụ đánh giá tự động đã chuẩn bị, cho ra các độ đo RMSE/MAE/CEP/P95/Max theo từng điều kiện truyền sóng (LOS/NLOS) và từng

khâu xử lý (thô/đã lọc). Bảng 4.5 là khung kết quả đã được dựng sẵn với cấu trúc trùng khớp khung kiểm chứng ở Bảng 4.3; các ô đánh dấu "—" sẽ được điền trực tiếp bằng số liệu đo thực địa mà không cần thay đổi cấu trúc bảng.

Bảng 4.5: Kết quả đo định vị thực địa (khung chờ số liệu — sẽ cập nhật khi có dữ liệu đo)

Nhóm	n	RMSE (cm)	MAE (cm)	CEP (cm)	P95 (cm)	Max (cm)
LOS / đã lọc	—	—	—	—	—	—
LOS / thô	—	—	—	—	—	—
NLOS / đã lọc	—	—	—	—	—	—
NLOS / thô	—	—	—	—	—	—
TỔNG — thô	—	—	—	—	—	—
TỔNG — đã lọc	—	—	—	—	—	—

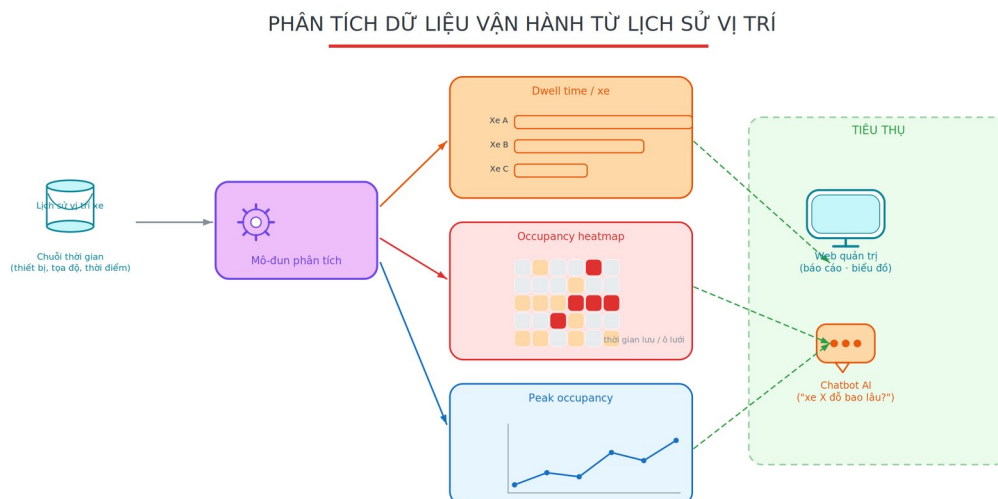
Ghi chú: bảng trên hiện để trống có chủ đích — đây là khung kết quả chờ dữ liệu đo thực địa. Khi tiến hành đo, mỗi điểm tham chiếu được ghi đồng thời vị trí thô và vị trí đã lọc trong hai điều kiện LOS/NLOS, sau đó điền các giá trị tương ứng vào bảng. Giá trị kỳ vọng theo tài liệu được nêu ở Bảng 4.4; kết quả kiểm chứng quy trình trên dữ liệu mô phỏng ở Bảng 4.3.

4.8 Phân tích và khai thác dữ liệu vận hành

Toàn bộ lịch sử vị trí phương tiện được lưu vào bảng chuỗi thời gian dữ liệu lịch sử vị trí phương tiện, tạo nền tảng cho việc phân tích vận hành ngoài chức năng định vị thời gian thực. Hệ thống bổ sung một mô-đun phân tích (AnalyticsService) trên IoT Server, cung cấp ba nhóm chỉ tiêu được tính trực tiếp từ lịch sử vị trí và lộ ra qua REST API theo từng tầng:

- **Thời gian lưu bãi (dwell-time) theo phương tiện:** tổng thời gian mỗi xe hiện diện trên tầng, được tách thành các phiên đỗ — khoảng cách thời gian giữa hai bản ghi liên tiếp vượt ngưỡng (mặc định 5 phút) được coi là xe đã rời đi rồi quay lại, nhờ đó thời gian vắng mặt không bị tính vào thời gian đỗ.
- **Bản đồ nhiệt mức độ sử dụng (occupancy heatmap):** chia mặt bằng thành lưới ô (mặc định 0,5m), mỗi ô tích lũy tổng thời gian phương tiện lưu lại trong phạm vi ô đó; phần đóng góp của mỗi bước thời gian được giới hạn trần để một khoảng trống dữ liệu đơn lẻ không chi phối kết quả. Bản đồ nhiệt giúp nhận diện các khu vực "nóng" phục vụ tối ưu hóa bố trí ô đỗ.
- **Lưu lượng đỉnh (peak occupancy):** số phương tiện khác nhau hiện diện trong từng khung thời gian (mặc định 5 phút) cùng giá trị đỉnh toàn cục, phục vụ báo cáo công suất sử dụng bãi.

Các phép tính được tách thành hàm thuần (không phụ thuộc cơ sở dữ liệu) nên kiểm thử đơn vị được độc lập, bảo đảm tính đúng đắn của thuật toán tổng hợp. Kết quả phân tích được giao diện web quản trị sử dụng để hiển thị báo cáo và biểu đồ, đồng thời là nguồn dữ liệu để chatbot trả lời các truy vấn mang tính thống kê như "khu vực nào hay được sử dụng nhất?" hoặc "xe X đã đỗ bao lâu?". Luồng xử lý từ lịch sử vị trí đến ba nhóm chỉ tiêu và phía tiêu thụ được mô tả trong Hình 4.6.



Hình 4.6: Quy trình phân tích dữ liệu vận hành

4.9 Đánh giá hợp đồng công cụ – thẻ giao diện

Bên cạnh đường RAG, chatbot còn sinh thẻ giao diện trực quan từ kết quả công cụ MCP. Khóa luận kiểm thử hợp đồng xác định giữa tầng công cụ và tầng hiển thị (ứng dụng di động): nạp dữ liệu mẫu cố định vào bộ dựng thẻ giao diện rồi kiểm tra tính hợp lệ của lược đồ dữ liệu, sự tồn tại bộ hiển thị tương ứng phía ứng dụng, và độ khớp tham số.

Bảng 4.6: Kết quả kiểm thử hợp đồng công cụ – thẻ giao diện

Độ đo	Giá trị
Tỷ lệ hợp lệ của lược đồ dữ liệu	8/8 (100%)
Tỷ lệ thẻ thiếu bộ hiển thị trên ứng dụng	0% (0/8)
Khớp tham số (trường bắt buộc đúng kiểu)	8/8 (100%)
Tính hợp lệ khi hiển thị	8/8 (100%)
Độ phủ kiểm thử (công cụ có kiểm thử / tổng)	8/8 (100%)

Kết quả kiểm thử cho thấy cả 8 ánh xạ công cụ → thẻ giao diện (7 loại thẻ phân biệt; ứng dụng di động có 12 loại bộ hiển thị) đều hợp lệ về lược đồ dữ liệu, có bộ hiển thị tương ứng và khớp tham số — tầng trình bày không có lỗi hợp đồng nào. Quá trình kiểm thử còn phát hiện 5 loại thẻ giao diện "dự phòng" (đã có bộ hiển thị nhưng chưa có công cụ nào sinh ra) — đây là bộ khung giao diện sẵn sàng cho các tính năng tương lai mà không cần thay đổi phía ứng dụng di động.

Kết luận chương 4. Chương 4 đã trình bày kết quả hiện thực hóa hệ thống. Ở lớp IoT và định vị: các nhóm tính năng ứng dụng di động (bản đồ 3D isometric thời gian thực, quản lý xe, tìm xe qua bản đồ UWB và giải pháp đề xuất Tag cầm tay+BLE, khảo sát mặt bằng walk-and-press, phát hiện đỗ sai, cảnh báo an ninh geofence/chống trộm, đề xuất bố trí tự động, chế độ Standard/Valet, thông báo đẩy), giao diện web quản trị IoT Admin, chuỗi pipeline định vị 5 tầng (firmware 1 Hz → trilateration stateless → One-Euro minCutoff=0,5/ β =0,1 → throttle 10 cm/30 s → Redis TTL 5 s

→ SSE), phương pháp đánh giá định lượng (RMSE/MAE/CEP/P95, kết quả mô phỏng §4.5.4, công cụ tái lập tự động), và kiểm chứng runtime (8 tag/5 anchor/mặt bằng hình L, 0 lỗi JS §4.5.6). Ở lớp AI: chatbot tiếng Việt (MCP+RAG đạt 100% trên n=101 câu, chấm khách quan) với vòng lặp agentic và dự phòng đa nhà cung cấp. Ngoài ra, mô-đun phân tích dữ liệu vận hành khai thác lịch sử vị trí để phục vụ báo cáo và truy vấn qua chatbot. Việc đo đạc định lượng trên phần cứng thực được tiến hành theo phương pháp đã chuẩn hóa ở Mục 4.5.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Qua quá trình nghiên cứu và triển khai thực nghiệm, khóa luận đã hoàn thành các mục tiêu đề ra và xây dựng được một hệ thống giám sát gara thông minh hoàn chỉnh dựa trên công nghệ AIoT, lấy định vị UWB làm trọng tâm. Ở lớp định vị nền tảng, đề tài đã hiện thực hóa trọn vẹn chuỗi xử lý thời gian thực từ bản tin khoảng cách Tag–Anchor đến tọa độ phương tiện, gồm giải Trilateration bằng bình phương tối thiểu và làm trơn quỹ đạo bằng bộ lọc One-Euro; đồng thời chuẩn hóa một quy trình đánh giá định lượng độ chính xác (RMSE/MAE/CEP/P95) kèm công cụ tái lập tự động (Mục 4.6), trong đó kết quả đo thực địa đang được tiến hành. Trên nền định vị đó, hệ thống được phát triển thành một kiến trúc đầu–cuối liền mạch, trải từ phần cứng nhúng ESP32-C3/EWT550, qua hạ tầng MQTT – NestJS – Redis – PostgreSQL, đến ứng dụng di động và giao diện quản trị web.

Bên cạnh lớp định vị, khóa luận tích hợp thành công một trợ lý hỏi đáp tiếng Việt ứng dụng kỹ thuật Hybrid RAG như một tiện ích bổ trợ ở lớp trí tuệ nhân tạo. Hiệu quả của trợ lý được kiểm chứng khách quan trên bộ 101 câu hỏi: cấu hình MCP+RAG đạt độ chính xác 100% (khoảng tin cậy Wilson 95%: 96–100%) khi chấm tự động bằng đối chiếu dữ kiện mà không dùng mô hình ngôn ngữ làm giám khảo (§4.3.7); ở khâu truy hồi, cấu hình Hybrid đạt $\text{Recall}@5 = 0,821$ và $\text{MRR} = 0,962$ (§4.3.6). Ngoài ra, hệ thống còn bổ sung một mô-đun phân tích dữ liệu vận hành — gồm thời gian lưu bãi, bản đồ nhiệt mức độ sử dụng và lưu lượng đỉnh — khai thác lịch sử vị trí để phục vụ báo cáo và truy vấn qua chatbot (Mục 4.7). Nhìn chung, hệ thống đã vận hành thông suốt trên môi trường thực và giải quyết được bài toán quản lý bãi đỗ xe thông minh đặt ra ban đầu; việc kiểm chứng định lượng độ chính xác định vị trên hệ thống thực đang được hoàn tất theo phương pháp đã chuẩn hóa ở Mục 4.6.

5.2 Hạn chế của đề tài

Mặc dù đạt được những kết quả khả quan, hệ thống vẫn còn một số hạn chế cần được nhìn nhận thẳng thắn. Trước hết, độ chính xác định vị vẫn chịu ảnh hưởng

đáng kể từ các vật cản kim loại lớn — vốn là đặc trưng của môi trường bãi đỗ xe — gây ra hiện tượng che khuất tầm nhìn (NLOS) làm suy giảm và phản xạ tín hiệu. Bên cạnh đó, chi phí phần cứng UWB tuy đã thấp nhưng vẫn cao hơn so với các công nghệ dựa trên cường độ tín hiệu như BLE hay Wi-Fi, phần nào ảnh hưởng đến khả năng nhân rộng ở quy mô rất lớn. Hệ thống cũng mới chỉ được thử nghiệm với số lượng Tag và Anchor hạn chế trên mô hình một tầng, nên hành vi ở quy mô nhiều tầng và mật độ phương tiện cao còn cần được kiểm chứng thêm. Cuối cùng, đề tài chưa hoàn tất quá trình đo đạc định lượng độ chính xác trên hệ thống thực; các số liệu RMSE/MAE/CEP cùng độ trễ đầu-cuối sẽ được bổ sung theo phương pháp đã chuẩn hóa ở Mục 4.6 ngay khi điều kiện phần cứng và mặt bằng cho phép.

5.3 Hướng phát triển tương lai

Từ những kết quả và hạn chế nêu trên, đề tài xác định một số hướng phát triển tiếp theo. Hướng đầu tiên là mở rộng bài toán định vị từ hai chiều (x, y) sang ba chiều (x, y, z) để phục vụ các bãi đỗ xe nhiều tầng, nơi thông tin độ cao trở nên cần thiết. Hướng thứ hai là tối ưu hóa thuật toán và lập lịch truyền tin khi số lượng phương tiện tăng lớn, nhằm sử dụng hiệu quả băng thông UWB và duy trì độ trễ thấp ở mật độ cao. Hướng thứ ba là tích hợp thêm các cảm biến môi trường như khói và nhiệt độ để nâng cao tính an toàn cho gara, qua đó mở rộng hệ thống từ giám sát vị trí sang giám sát an toàn toàn diện. Ngoài ra, ở lớp trợ lý AI, việc hoàn tất đo đạc thực địa và khảo sát trên truy vấn của người dùng thật sẽ giúp củng cố thêm tính tổng quát của các kết quả đánh giá.

TÀI LIỆU THAM KHẢO

1. IEEE, "IEEE Std 802.15.4z-2020 — IEEE Standard for Low-Rate Wireless Networks, Amendment 1: Enhanced Ultra Wideband (UWB) Physical Layers (PHYs) and Associated Ranging Techniques," IEEE, Aug. 2020. doi: 10.1109/IEEESTD.2020.9179124.
2. G. Casiez, N. Roussel, and D. Vogel, "1€ Filter: A Simple Speed-based Low-pass Filter for Noisy Input in Interactive Systems," in Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI '12), Austin, TX, USA, 2012, pp. 2527–2530. doi: 10.1145/2207676.2208639.
3. P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proc. 34th Conf. Neural Information Processing Systems (NeurIPS), Vancouver, Canada, 2020. arXiv:2005.11401.
4. NestJS Documentation. [Online]. Available: <https://docs.nestjs.com>
5. Model Context Protocol (MCP) Specification. [Online]. Available: <https://modelcontextprotocol.io>
6. F. Zafari, A. Gkelias, and K. K. Leung, "A Survey of Indoor Localization Systems and Technologies," IEEE Communications Surveys & Tutorials, vol. 21, no. 4, pp. 2568–2599, 2019. doi: 10.1109/COMST.2019.2911558.
7. F. Che, Q. Z. Ahmed, P. I. Lazaridis, P. Sureephong, and T. Alade, "Indoor Positioning System (IPS) Using Ultra-Wide Bandwidth (UWB) — For Industrial Internet of Things (IIoT)," Sensors, vol. 23, no. 12, art. 5710, 2023. doi: 10.3390/s23125710.
8. OASIS, "MQTT Version 5.0," OASIS Standard, A. Banks, E. Briggs, K. Borgendale, and R. Gupta, Eds., 7 Mar. 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
9. G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in Proc. 32nd

- Int. ACM SIGIR Conf. Research and Development in Information Retrieval (SIGIR '09), Boston, MA, USA, 2009, pp. 758–759. doi: 10.1145/1571941.1572114.
10. S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009. doi: 10.1561/15000000019.
 11. D. H. Douglas and T. K. Peucker, "Algorithms for the reduction of the number of points required to represent a digitized line or its caricature," *Cartographica*, vol. 10, no. 2, pp. 112–122, 1973. doi: 10.3138/FM57-6770-U75U-7727.
 12. R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960. doi: 10.1115/1.3662552.
 13. Espressif Systems, "ESP32-C3 Series Datasheet," Espressif Systems, 2024. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-c3_datasheet_en.pdf
 14. Anthropic, "Introducing Contextual Retrieval," Anthropic, Sep. 2024. [Online]. Available: <https://www.anthropic.com/news/contextual-retrieval>